

Functional Programming Paradigm

11

In this chapter, we discuss the functional programming paradigm, where computation proceeds by term rewriting and not through modification of the state. Languages of this paradigm, at least in their “pure” form, do not use the concept of memory (and therefore there is no side effect). Once an environment is fixed, an expression always denotes the same value.

We will discuss the pure paradigm in the first sections, explaining its fundamental aspects. Real functional programming languages, however, merge these “pure” ingredients with many other mechanisms; we will review some of them in Sect. 11.3. Also most modern imperative languages incorporate constructs for a functional programming style. We will conclude this first part of the chapter discussing the reasons why the functional programming paradigm has gained much interest in the last decades.

We will touch on the SECD machine, an abstract machine for higher-order functional languages which is the prototype of many real implementations.

The chapter concludes with a more theoretical section that provides a succinct introduction to the λ -calculus, a formal system for computability that inspires all functional languages and which has, since the time of ALGOL and LISP, been a constant model for the design of programming languages.

11.1 Computing Without State

Conventional languages base their computational model on the transformation of the *state*. The heart of this model is the concept of *modifiable variable*, that is, a container with a name to which, during the computation, can be assigned different values, while the same association is always maintained in the environment. Correspondingly, the principal construct of conventional languages is *assignment*, which modifies the value of a variable, without changing the association between the name of the

variable and the location to which it corresponds. In the terminology of Sect. 6.2.2, it modifies the r-value but not the l-value, which is fixed when the variable is declared. This conventional, “imperative” computational model is an abstract view of the underlying physical machine, where computation proceeds by modifying values stored in locations, according to the so-called *von Neumann Machine* model, named after the Hungarian-American mathematician who, in the 1940s, was one of the prominent designers of one of the first electronic stored-program computers.

In pure functional languages, there is neither a state nor a modifiable variable. The computation proceeds—at least in principle—by *rewriting* expressions, by changes that take place only in the environment and do not involve the concept of memory. If there are no modifiable variables, there is no longer a need for assignment. The entire computation is expressed in terms of modifications to the environment in which the manipulation of (higher-order, see Sect. 7.2) functions plays a fundamental role.

Without assignment, iteration becomes less important. A loop repeatedly modifies the state until the values of certain variables satisfy a guard. The reader already knows (see Chap. 6) that iterative and recursive constructs are two equivalent mechanisms permitting infinitely long (and possibly divergent) computations. In the stateless computation model, (unbounded) iteration disappears, recursion remains, becoming the fundamental construct for sequence control. Bounded iteration is expressed via higher-order functions, see Sect. 11.3.4.

Higher-order functions and recursion are the basic ingredients of this stateless computational model, which is called the *functional programming paradigm*, as opposed to the ordinary, state based *imperative paradigm*.

This is a paradigm as old as the imperative one. Since the 1930s, beside the Turing Machine (see box on Sect. 3.4), there has existed the λ -calculus, an abstract model for computable functions based on those same concepts we have been briefly explaining. LISP was the first programming language explicitly inspired by the λ -calculus and many others have followed in the intervening years (Scheme, ML, in all its different dialects, Miranda, Haskell, Erlang to name just a few of them). Among these, only Miranda and Haskell are “purely functional”; the others also have imperative components, although these languages assign a subordinate role to them.

We introduce first the pure functional paradigm. To make the treatment more concrete, we refer to a specific language rather than using a neutral pseudocode. ML has been chosen for this because, through its coherence and elegance of design, it is the most suited of them all to didactic presentation. It is not our aim to introduce the language (for which, the reader should refer to the bibliography), but only to use the syntax in an instrumental fashion to discuss some general questions that apply also to other functional languages.

11.1.1 Expressions and Functions

In the usual mathematical practice, there is some ambiguity about when we are defining a function and when we are applying it to a value. It is not uncommon to encounter expressions of the kind:

Let $f(x) = x^2$ be the function that associates with x its square. If now we have $x = 2$, it follows that $f(x) = 4$.

The syntactic expression $f(x)$ is used to denote two things that are quite different: the first time, it serves to introduce the *name*, f , of a specific function; the second time, it serves to denote the result of *applying* the function f to a specified value. In mathematical practice, this ambiguity is completely innocuous because the context helps us to distinguish which of the two uses is intended. The same does not hold for an artificial language (such as a programming language) which describes functions. Here, it is appropriate to distinguish the two cases.

When a mathematician asserts that they are defining the function, $f(x)$, in reality, they are defining the function f with one *formal* parameter, x , which serves to indicate the transformation that f applies to its argument. To distinguish linguistically between the name and the “body” of the function, following ML syntax, we can write:

```
val f = fn x => x*x;
```

The reserved word `val` introduces a declaration. The declaration is used to extend the environment with a new association between a name and a value. In our case, the name f is bound to the transformation of x into $x * x$. In all functional languages, functions are *expressible* values; that is, they can be the result of the evaluation of a complex expression. In our case, the expression on the right of the `=` and introduced by `fn` is an expression that denotes a function.

For the application of a function to an argument, we retain the traditional notation, writing $f(2)$ or $(f\ 2)$, or $f\ 2$, for the expression that results from the application of a function f to the argument 2. We can also use `val` to introduce new names, as in:

```
val four = f 2;
```

The introduction of specific syntax for an expression which denotes a function has an important consequence. It is possible to write (and possibly apply) a function without having necessarily to assign it a name. For example, the expression

```
(fn y => y+1) (6);
```

has the value 7 which results from the application of the (anonymous) function `fn y => y+1` to the argument 6. To make the notation less imposing, we assume (as does ML) that application can be denoted by simple juxtaposition (that is without parentheses) and that it associates to the left (prefix notation). If g is the name of a function,

```
g a1 a2 ... ak
```

will mean:

```
((...((g a1) a2)... ak)).
```

Nothing prevents one functional expression from appearing inside another, as in

```
val add = fn x => (fn y => y+x);
```

The value `add` is a function which, given an argument, `x`, returns an anonymous function which, given an argument `y`, returns `x+y`. We can use `add` in many different ways:

```
val three = add 1 2;
val addtwo = add 2;
val five = addtwo 3;
```

Note that, in particular, `addtwo` is a function which is obtained as the result of the evaluation of another expression.

The notation which uses `val` and `fn` is important but is a little verbose. ML also allows the use of a more compact notation which resembles the usual way of defining functions in a programming language. The first function, `f`, which we defined (the one which computes the square of its argument) could also be defined as follows:

```
fun f x = x*x;
```

In general, a definition of the form:

```
fun F x1 x2 ... xn = body;
```

is only syntactic sugar (that is, in programming language jargon, only a nicer abbreviation) for:

```
val F = fn x1 => (fn x2 => ... (fn xn => body) ...);
```

As a last example of functions which manipulate functions, this time in the form of a formal parameter, consider the definition:

```
fun comp f g x = f(g(x));
```

This returns the function composed of its first two arguments (which are, in their turn, functions).

Finally, every functional program permits the definition of recursive functions. Assuming that we have available a *conditional expression* (which in ML is written with the familiar `if then else` syntax), we can define the usual factorial as:

```
fun fatt n = if n=0 then 1 else n*fatt(n-1);
```

11.1.2 Computation as Reduction

If we exclude arithmetic functions (which we can assume predefined with the usual semantics) and the conditional expression, at the conceptual level, we can describe the procedure used to transform a complex expression into its value (*evaluation*) as the process of *rewriting*. We call this process *reduction*. In a complex expression, a subexpression of the form “function applied to an argument” is textually replaced by the body of the function in which the formal parameter is replaced, in its turn, by the

```

fun K x y = x;
fun S p q r = p r (q r);
val a = ...;

S K K a → (fn p => (fn q => (fn r => p r (q r)))) K K a
        → (fn q => (fn r => K r (q r))) K a
        → (fn r => K r (K r)) a
        → K a (K a)
        → (fn x => (fn y => x)) a (K a)
        → (fn y => a) (K a)
        → a

```

Fig. 11.1 Some definitions and a computation using rewriting

actual parameter.¹ We can compute a simple expression using this computational model (we use $\rightarrow$ to indicate a reduction step).

```

fact 3 → (fn n => if n=0 then 1 else n*fact(n-1)) 3
        → if 3=0 then 1 else 3*fact(3-1)
        → 3*fact(3-1)
        → 3*fact(2)
        → 3*((fn n => if n=0 then 1 else n*fact(n-1)) 2)
        → 3*(if 2=0 then 1 else 2*fact(2-1))
        → 3*(2*fact(2-1))
        → 3*(2*fact(1))
        → 3*(2*((fn n => if n=0 then 1 else n*fact(n-1)) 1))
        → 3*(2*(if 1=0 then 1 else 1*fact(1-1)))
        → 3*(2*(1*fact(0)))
        → 3*(2*(1*((fn n => if n=0 then 1 else n*fact(n-1)) 0)))
        → 3*(2*(1*(if 0=0 then 1 else n*fact(n-1))))
        → 3*(2*(1*1))
        → 6

```

Note how, with the exception of arithmetic calculations and the use of the conditional expression, all of the rest of the computation proceeds by symbolic manipulation of strings: no variables, no update of values on the execution stack. Figure 11.1 contains another example of pure symbolic manipulation. (It is a complicated way to write the identity function!) (While studying the figure, recall that `fn` is just an abbreviation for `val ... fn`.)

Finally, the reader will have no difficulty in convincing themselves that, given the definition

```

fun r x = r(r(x));

```

each computation that involves an evaluation of r resolves into an infinite rewriting. We say, in such a case, that the computation *diverges* and that the result is undefined.

¹ The reader will certainly have recognised in this description the “copy rule” we already discussed in the context of the by-name parameter passing. For the time being, we will be vague about the exact semantics to assign to this process—all of Sect. 11.2 will be devoted to it.

11.1.3 The Fundamental Ingredients

In these first subsections, we have introduced the fundamental ingredients of the pure functional paradigm. We can make this precise and summarise the main concepts in the following way.

From the syntactic viewpoint, a pure functional language has no commands (there being no state to modify using side effects) but only expressions. Apart from possible values and primitive operators for data (such as integer, boolean, characters, etc.) and the conditional expression, the two principal constructs for defining expressions are:

- *Abstraction* which, given any expression, exp and an identifier, x , allows the construction of an expression $\text{fn } x \Rightarrow \text{exp}$ denoting the function that transforms the formal parameter x into exp (the expression exp is “abstracted” from the specific value bound to x).
- The *application* of an expression, f_exp , to another expression, a_exp , which we write $f_exp \ a_exp$, which denotes the application of the function (denoted by) f_exp to the argument (denoted by) a_exp .

There are no constraints on the possibilities of passing functions as arguments to other functions, or to returning functions as the results of other (higher-order) functions. As a consequence, there is perfect *homogeneity* between programs and data.

From the semantic viewpoint, a program consists of a series of value definitions, each of which inserts a new association into the environment and can require the evaluation of arbitrarily complex expressions. The presence of higher-order functions and the possibility of defining recursive functions makes this definition mechanism flexible and powerful.

To a first approximation, the semantics of computation (*evaluation*) refers to no linguistic aspects other than the ones introduced so far. It can be defined using simple symbolic rewriting of strings (*reduction*), which repeatedly uses two main operations to simplify expressions until they reach a simple form which immediately denotes a value. The first of these operations is the simple search through the environment. When an identifier is determined as being bound in an environment, replace the identifier by its definition. For example, in Fig. 11.1, we used this operation for the first and the fifth reduction. In what follows this step will not be explicitly considered again. We will consider a name as a simple abbreviation for the value associated with it.

The second operation, which is more interesting, deals with a functional expression applied to an argument and uses a version of the copy rule (which in this context is called the β -rule).

Definition 11.1

Redex A *redex* (which stands for a *reducible expression*) is an application of the form $((\text{fn } x \Rightarrow \text{body}) \ \text{arg})$.

Reductum The *reductum* of a redex $((\text{fn } x \Rightarrow \text{body}) \text{ arg})$ is the expression which is obtained by replacing in *body* each (free) occurrence of the formal parameter, x , by a copy of *arg* (avoiding variable capture,² see Sect. 7.1.2).

β -rule An expression, *exp*, in which a redex appears as a subexpression is reduced (or rewrites, simplifies) to *exp1* (notation: $\text{exp} \rightarrow \text{exp1}$), where *exp1* is obtained from *exp* by replacing the redex by its reductum.

From an implementation viewpoint, finally, every abstract machine for functional languages adopts extensive garbage-collection mechanisms because, in the case in which functions are returned as values by other functions, we know that the local environment must be preserved for an unlimited period of time (see Sect. 7.2.2).

If these are the cardinal notions of a functional language, on a more attentive reading, these few lines raise more problems than they solve. We undertake a deeper analysis of these concepts in the next section.

11.2 Evaluation

Last section leaves open at least two issues:

- What is the termination condition for reduction (that is what does “a simple form which immediately denotes a value” mean?)
- What precise semantics must be given to the β -rule, not just concerning possible variable capture (which we already described in the context of parameter passing) but more importantly, the order to follow during rewriting, should more than one redex be present in the same expression?

11.2.1 Values

A *value* is an expression which cannot be further rewritten. In a functional language, there are values of two kinds: values of primitive type and functions. There is little to say about values of simple types. If the language provides some primitive types (integers, booleans, characters, etc.) is clear that each of these types come with a set of values that do not admit evaluation (for example, constants of type integer, boolean, characters, etc.). In the example of the previous subsection, we terminated the evaluation of *fact* when we reached primitive values of integer type: 3, 2, 1.

²In the context of functional programming, identifiers bound to values are often referred to as “variables”. The reader is by now experienced enough to allow us to continue with this traditional use without being confused by the absence of modifiable variables in pure functional programming languages.

Functional values are more interesting. Let us consider the following definition:

```
val G = fn x => ((fn y => y+1) 2);
```

We have said that a definition entails the evaluation of the expression on the right of the equality and, then, the binding of the value thus derived to the name on the left of the `=`. But, in this case, it is not immediately clear what the value to be associated with `G` is. Do we have:

```
fn x => 3
```

in which we have rewritten the body of `G` by evaluating the redex it contains, or do we have

```
fn x => ((fn y => y+1) 2)
```

in which there has been no evaluation in the body of `G`? The first case appears to be the one that more respects the informal semantics that we gave at the end of the last subsection. The second, on the other hand, is closer to the usual meaning of a function definition in imperative languages, in which the body of a function is only evaluated when it is called.

Functional languages in common use all adopt the latter of these choices—there is no evaluation “under” an abstraction. Every expression of the form

```
fn x => exp
```

is a value, so redexes possibly contained in `exp` are *never* rewritten until the function is applied to some argument.

11.2.2 Capture-Free Substitution

To implement capture-free substitution, we saw in Chap. 7 that closures can be used. This is, in effect, the mechanism also used by abstract machines for functional languages (see Sect. 11.5). For the elementary description we are giving at present, though, a syntactic convention suffices. In every expression, there are never two formal parameters with the same name and the names of the possible variables that are not formal parameters are all distinct from those of the formal parameters.

Using this convention, there will never be variable capture in the simple examples that we consider.³

11.2.3 Evaluation Strategies

In Chap. 6, while discussing expressions, we saw how every language must fix a specific strategy (that is a fixed order) for the evaluation of expressions. The presence

³ In the more general case, if we do not want to bring into play the concept of closure, for the correct description of computation using rewriting, it is necessary to carefully define the concepts of bound and free variable, and substitution, all concepts which we treat formally in Sect. 11.6.

Fig. 11.2 An expression with more than one redex

```

fun K x y = x;
fun r z = r(r(z));
fun D u = if u=0 then 1 else u;
fun succ v = v+1;

val v = K (D (succ 0)) (r 2);

```

of higher-order functions in functional languages makes this question even more fundamental. To clarify the problem, consider the definitions in Fig. 11.2. Which value is associated with v and how is it determined?

The β -rule on its own is not of much use because, in the right-hand part of the definition of v , there are 4 redexes (after the expansion of the names with the values associated with them by the definitions):

```

K (D (succ 0))
D (succ 0)
succ 0
r 2

```

Which of them is reduced first? Every commonly used language uses a leftmost strategy which reduces the redexes starting with the one at the leftmost end. But even having stipulated this, it is not clear which is the leftmost redex among

```

K (D (succ 0))
D (succ 0)
succ 0

```

because these three redexes are superimposed on each other. Having fixed a leftmost evaluation order, there are three main different strategies.

Evaluation by Value

In evaluation by value (which is also called applicative-order, or *eager*, or *innermost* evaluation), a redex is evaluated only if the expression which constitutes its argument part is already a value. More precisely, leftmost evaluation in applicative order works as follows.

1. Scan the expression to be evaluated from the left, choosing the first application encountered. Let it be $(f_exp\ a_exp)$.
2. First evaluate (recursively applying this method) f_exp until it has been reduced to a value (of functional type) of the form $(fn\ x\ =>\ \dots)$.
3. Then evaluate the argument part, a_exp , of the application, so that it is reduced to a value, val .
4. Reduce the redex $((fn\ x\ =>\ \dots)\ val)$ using the β -rule and repeat from (1).

Considering Fig. 11.2, case (1) first chooses the application $K\ (D\ (succ\ 0))$. Now some elementary applications of (1), (2) and (3) will serve to show that K , D and $succ$ are already values (recall that we are implying that a name is an abbreviation for the expression associated with it). The first redex to be reduced is then $succ\ 0$ (that

is, $((\text{fn } v \Rightarrow v + 1) \ 0))$ which will be completely evaluated and yields 1. Then the redex $(D \ 1)$ (that is $((\text{fn } u \Rightarrow \text{if } u=0 \text{ then } 1 \text{ else } u) \ 1))$) is reduced to give the value 1. Then $(K \ 1)$ is evaluated to produce the value $(\text{fun } y \Rightarrow 1)$. At this point in the evaluation, the expression has become

```
(fun y => 1) (r 2)
```

Since the functional part of this application is already a value, the strategy prescribes that the argument $(r \ 2)$ is evaluated. An application of the β -rule leads to $r \ (r \ 2)$; we then proceed to $r \ (r \ (r \ 2))$ and so on in a divergent computation.

No value is therefore associated with v because the computation diverges.

Evaluation by Name

In the evaluation by name strategy (which is also called *normal order* or *outermost* evaluation), a redex is reduced without first reducing its argument part. More precisely, leftmost evaluation in normal order proceeds as follows.

1. Scan the expression to be evaluated from the left, choosing the first application encountered. Let it be $(f_exp \ a_exp)$.
2. First evaluate f_exp (recursively applying this method) until it has been reduced to a value (of functional type) of the form $(\text{fn } x \Rightarrow \dots)$.
3. Reduce the redex $((\text{fn } x \Rightarrow \dots) \ a_exp)$ using the β -rule and repeat from (1).

Considering Fig. 11.2, the first redex to be reduced is therefore:

```
K (D (succ 0))
```

It is rewritten to:

```
fn y => D (succ 0)
```

which is a functional value. The full expression now is of the form:

```
(fn y => D (succ 0)) (r 2)
```

for which the strategy prescribes reducing the outermost redex, so we obtain:

```
D (succ 0)
```

Now reducing this expression, we obtain:

```
if (succ 0)=0 then 1 else (succ 0)
```

and then:

```
if 1=0 then 1 else (succ 0)
```

and finally

```
succ 0
```

from which we obtain the final value, 1, which is then associated with v .

Lazy Evaluation

In evaluation by name, a single redex might have to be evaluated more than once because of some duplication that has occurred during rewriting. In the example that we are discussing, the redex `(succ 0)` is duplicated because of the function `D` and is reduced twice⁴ in the conditional expression that forms the body of `D`. This is the price that must be paid for postponing the evaluation of an argument until after the application of a function (and it is really this that allows evaluation by name to obtain a value where evaluation by value would diverge). But it is expensive in terms of efficiency (since the duplicated redex may require a significant amount of computation).

To obviate this problem and maintain the advantages of evaluation by name, the lazy strategy proceeds like that by name but the first time that a “copy” of a redex is encountered, its value is saved and will be used should any other copies of the same redex be encountered. Since there is no state, the evaluation of multiple copies of the same expression will always terminate with the same value, hence guaranteeing that the lazy strategy computes the same value of the strategy by name.

The by-name and lazy strategies are examples of *call by need* strategies in which a redex is reduced only if it is required by the computation.

11.2.4 Comparison of the Strategies

In the last subsection, the by-name strategy produces a value when the by-value strategy diverges. It is sensible to ask the question as to whether it can be the case that the two strategies produce *distinct* values for the same expression.

An answer to this question is given by the following theorem which expresses one of the most important properties of the *pure* functional paradigm. We say that an expression in the language is *closed* if all of its variables are bound by some `fn`. Let us recall, then, from Sect. 11.2.1, that by *primitive value*, we mean a value of a primitive type (integer, boolean, character, etc.), excluding, therefore, functional values.

Theorem 11.1 *Let exp be a closed expression. If exp reduces to a primitive value, val , using any of the three strategies of Sect. 11.2.3, then exp reduces to val using the by-name strategy. If exp diverges using the by-name strategy, then it also diverges under the other two strategies.*

Let us note that the theorem excludes that a (closed) expression can yield a primitive value, `val`, in one strategy and yield *another* primitive value, `val2`, using

⁴The purist does not speak of copies of redexes but will rather say that the redex `(succ 0)` has given way to two *residuals*, each of which has been reduced independently of the other.

another strategy.⁵ Two strategies can therefore differ only by the fact that one determines a value while the other diverges, as we saw in our example. We cannot go into the details of how the theorem can be proved but it is important to stress that the following basic property is fundamental to its validity:

Once any strategy has been fixed in a given environment, the evaluation of all occurrences of a single expression always yields the same value.

This property, which is immediately falsified when there are side effects, is taken by many authors as the *criterion* for a pure functional language: a language is purely functional if it satisfies this condition. This is a very important property which makes it possible to *reason* about a functional program and to which we will return in Sect. 11.4. Let us again note, before going on, that it is just by virtue of this property that lazy evaluation is correct. The value obtained by the evaluation of an expression does not change when another copy of the same expression is encountered (clearly, in the meanwhile, the environment has not been changed).

In the light of the preceding theorem, it can be asked what interest there is in the by-value strategy, given that by-name is the most general of all possible strategies. Moreover, reasons of efficiency would seem to suggest that we only adopt call-by-need strategies, given that it is not clear why useless redexes should be reduced, with the consequent waste of computing time. The point is that the efficiency case is not so clear-cut. The implementation of a call-by-need strategy is, in general, more expensive than a simple call-by-value strategy. The by-value strategy has efficient implementations on conventional architectures, even if, sometimes, it performs unnecessary work (every time that an argument that is not required in the body of the function is evaluated, as happens with the second argument of the function K in Fig. 11.2). This last case, however, can be treated in an efficient manner with abstract evaluation techniques that attempt to identify useless arguments. Moreover, in functional languages including also imperative constructs, a by-value reduction strategy blends better with those imperative features.

Among the functional languages that we have cited at the beginning of this chapter, LISP, Scheme, ML, Erlang use a by-value strategy (also because they include major imperative aspects), while Miranda and Haskell (which are pure functional languages) use lazy evaluation.

⁵ The theorem is no longer valid if the hypothesis that the value is *primitive* is removed. Having assumed that evaluation is not performed under abstraction, in fact, the two strategies can yield (functional) values that are distinct. By way of an example, define $I = \lambda x. x$ and $P = \lambda x. \lambda y. (\lambda z. y z)$. The term $P (I I)$ reduces to $\lambda y. y (I I)$ under the by-name strategy, while, using the by-value strategy, it reduces to $\lambda y. y I$. This difference is not too important in programming languages (where we mainly want to compute values of primitive type). In Sect. 11.6, we will define an abstract calculus in which reduction also occurs in the presence of abstractions. This calculus satisfies the *confluence* property (stronger and more general than Theorem 11.1), which we will discuss on Sect. 11.6.

11.3 Programming in a Functional Language

The mechanisms we described in the last section are sufficient to express programs for every computable function (they constitute, indeed, a Turing-complete language). This is the core of every functional language, which, however, is too austere to be used as a real programming language. Every functional language, therefore, embeds this core in a wider context which provides mechanisms of different kinds, each aiming to make programming simpler and more expressive.

11.3.1 Local Environment

The mechanism of global definitions in the environment that we have used this far has too little structure for a modern language. As in conventional languages, it is appropriate to provide explicit mechanisms to introduce definitions with limited scope, as for example:

```
let x = exp in expl end
```

This introduces the binding of `x` to the value of `exp` in a scope which includes only `expl`.

To tell the truth, the presence of functional expressions already introduces nested scopes and associated environments which are composed of the formal parameters of the function bound to the actual parameters. From the point of view of evaluation, we can indeed consider the construct

```
let x = exp in expl end
```

as syntactic sugar for

```
(fn x => expl) exp.
```

The use of local scopes is too important, however, and justifies the introduction of the special syntax for them.

11.3.2 Interactiveness

Every functional language has an interactive environment. The language is used by entering expressions which the abstract machine evaluate, returning their values. Definitions are particular expressions which modify the global environment (and may return a value).

This model immediately suggests interpreter-based implementations, even if there are efficient implementations that generate compiled code the first time a definition is stored in the environment.

11.3.3 Pattern Matching

One of the most annoying aspects of recursive functional programming is handling terminal cases by means of explicit “if” expressions. Let us take, for example, the (very inefficient) function which returns the n th term of the Fibonacci series:

```
fun Fibo n = if n=0 then 1
             else if n=1 then 1
                   else Fibo (n-1)+Fibo (n-2);
```

The mechanism of *pattern matching* present in some languages such as ML and Haskell, allows us to give a definition (which is equivalent to the above) as follows:

```
fun Fibo 0 = 1
  | Fibo 1 = 1
  | Fibo n = Fibo (n-1)+Fibo (n-2);
```

The “|” character is read “or”. Each branch of the definition corresponds to a different case of the function. The most interesting part of this definition is the formal parameters. They are no longer constrained to be identifiers but can be *patterns*, that is, expressions formed from variables (the last is an example), constants (in the other cases) and other constructs that depend on the language’s type system (we will see an example using lists below). A pattern is a kind of schema, a model against which to match the actual parameter. When the function is applied to an actual parameter, it is compared with the patterns (using the order in which they appear in the program) and the body corresponding to the first pattern which matches with the actual parameter is chosen.

The pattern-matching mechanism is particularly flexible when structured types are used, for example lists.⁶ In ML syntax, a list is denoted by square brackets with the elements separated by commas. For example

```
["one", "two", "three"]
```

is a list of three strings. The operator, `::` denotes “cons” (that is the operator that adds an element to the head of a list):

```
"zero"::["one", "two", "three"]
```

is an expression whose value is the list:

```
["zero", "one", "two", "three"].
```

Finally, `nil` is the empty list. Thus

```
"four"::nil
```

has value `["four"]`. Using pattern matching, we can define a function which computes the length of a generic list such as:

⁶ Recall that in the functional realm, a list is an *immutable* data, thus what, e.g., in Python would be called instead a tuple.

```
fun length nil = 0
  | length e::rest = 1 + length(rest);
```

Note that the names used in the pattern are used as formal parameters to indicate parts of the actual parameter. It should be clear the advantage in terms of conciseness and clarity with respect to the usual definition:

```
fun length list = if list = nil then 0
                  else 1 + length(tl list);
```

This definition, moreover, requires the introduction of a *selection* function (which we have written `tl`) to obtain the list that is obtained by removing the head element. The selection operation is, on the other hand, implicit in pattern matching.

An important constraint is that a variable cannot appear twice in the same pattern. For example, consider the following “definition” of a function which, applied to a list, returns `true` if and only if the first two elements of the list are equal:

```
fun Eq nil = false
  | Eq [e] = false
  | Eq x::x::rest = true
  | Eq x::y::rest = false;
```

This definition is syntactically illegal because the third pattern contains the variable `x` twice, where it is used to test the equality of the first two elements. The pattern-matching mechanism is one way to check that the *form* of an actual parameter agrees with a pattern, not that the values it contains bear certain relationships. Otherwise stated, pattern matching is *not* unification, a much more general mechanism (and more complex to implement) and which we will discuss in the context of the logic programming paradigm (Chap. 12).⁷

11.3.4 Programming in a Functional Style

Programming in a *functional style* revolves around *immutable data*, manipulated by a (large) set of (small) functions. Using higher-order functions one may define general programming schemata as functions, which can then be instantiated to obtain specific program behaviors. A typical example is the so-called “*map, filter, fold pattern*” which is the functional equivalent of the “linear scan of a sequence” in imperative style.

The function `map(f, list_of_data)` is a (usually predefined) function that returns the list which is obtained by applying `f` to any element of `list_of_data`. Instead of the imperative Python code

⁷ The implementation of pattern matching is nothing other than the “obvious” translation into a sequence of `ifs`. In the case of `Eq`, the third pattern would require checking the equality of two elements of *arbitrary type*. If we have values for which the language does not define equality (as happens, for example, for functional values), it would not be possible to implement an equality test.

```
res=[]
for e in list_of_data:
    res.append(f(e))
```

we would simply write `res=map(f, list_of_data)`.

The function `fold(f, init, list_of_data)` is a (usually predefined) function that takes a binary function `f` and applies it to all the elements of `list_of_data`, returning the accumulated result; `init` is the initial value used to start the fold. For example, `fold(fn x,y=>x+y, 0, list_of_int)` returns the sum of all the elements of `list_of_int`.⁸

The function `filter(p, list_of_data)` is a (usually predefined) function that returns the list which is obtained by taking those elements of `list_of_data` which satisfy the predicate (the condition) `p`.

Other sets of higher-order functions allow for the concise (and efficient) expression of many other program schemata. The essential point we want to make is that the extensive use of program schemata increases the modularity of code. And the more modular a program is, the more easily we may factor the testing (or the verification, or the proof of correctness) of that program.

It is clear that one may use a *functional programming style* also using programming languages that allow for other programming paradigms. Once a language provides higher-order functions, it becomes easy to write large programs which avoid state-based computations. This is the case, for instance, of Scala, which has a large “functional subset” and has all the needed, predefined higher-order functions.⁹ Moreover, in Scala names can be introduced by two distinct constructs: `var` declares a name that can be re-assigned, while `val` binds a name to a value, without allowing any further modification of this binding. By using `val` and only unmodifiable values we avoid any side-effect, and hence we obtain a truly functional programming style. What is peculiar to *pure* programming languages is that the absence of any side-effect¹⁰ allows much easier control of the correctness of a program, see Sect. 11.4.

11.3.5 Types

As in conventional languages, the type system is an aspect of primary importance in functional languages, as well. Every functional language we have cited provides the usual primitive types (integers, booleans, characters) with operations on their values. With the exception of Scheme, which is a language with dynamic type checking, all the others have elaborate static type systems. These type systems allow the definition of new types such as pairs, lists, “records” (that is, tuples of labelled values). For

⁸ See the use of the anonymous function (the lambda) as argument to `fold`.

⁹ Also Python’s functions are first-class citizens. Differently from Scala, however, Python has no linguistic construct to limit the re-assignability of names. The isolation of a pure functional subset is therefore more problematic.

¹⁰ Or even the limited, confined side-effects of those languages, like Erlang and Scala, which are not pure but insist on encapsulating any side-effect into specific functions.

example, in ML, we can define a function `add_p` which takes as arguments a pair of integers and returns their sum:

```
fun add_p (n1, n2) = n1+n2;
```

Note the fundamental difference between `add_p` and `add` which we defined on Sect. 11.1.1: `add_p` requires a pair of integers and it would make no sense to provide just one value for `n1` and none for `n2`. Instead, `add` is a function which inputs *one* number and returns a function that takes another number and returns the sum of both.¹¹

An important part of the type system in functional languages is the one dedicated to the types of functions, given that functions are denotable and expressible values (and often also storable, if the language includes imperative aspects). In typed functional languages, this also means that some functional expressions are illegal because they cannot be typed. For example, the following higher-order function is illegal in most typed languages:

```
fun F f n = if n=0 then f(1) else f("pippo");
```

The reason for this is that the formal parameter, `f`, must simultaneously have types `int -> 'a` and `string -> 'a` (where `'a` denotes a type variable, that is, a generic type that is not yet instantiated).

Another illegal expression (because it violates typing) is self-application:

```
fun Delta x = x x;
```

The expression, `(x x)`, is illegal because there is no way to assign a unique and consistent type to `x`. Given that it occurs on the left as an application, it must have a type of the form `'a -> 'b`. Since, then, it also appears as the argument to a function (on the right of the application), it must have the type that the function requires: therefore, `x` must be of the type `'a`. Putting the two constraints together, we have it that `x` must, at the same time, be of type `'a` and of type `'a -> 'b` and there is no way to “unify” these two expressions.

In languages without a strong typing system, such as Scheme, the function `Delta` is, instead, legal. In Scheme, we can also apply `Delta` to itself. The expression (obviously written in Scheme syntax)

```
(Delta Delta)
```

constitutes a simple example of a divergent program. The lack of a static type check makes it possible in Scheme to write expressions such as:

```
(4 3)
```

Here, the aim is to apply the integer 4 to the integer 3. Since the left-hand part of this application is not a function, the abstract machine will generate an error at execution time.

¹¹ The passage from a function requiring, as argument, a pair (or, more generally, a tuple) to a unary higher-order function, is called the *curryfication* of a function. The operation is named after Haskell Curry, one of the founding fathers of the theory of the λ -calculus.

In the case of ML, the most interesting aspect of its type system is its support for polymorphism (see Sect. 8.8 and, in particular, the box on Sect. 8.7.4).

11.3.6 Infinite Objects

In the presence of by-name or lazy strategies, it is possible to define and manipulate *streams*, that is, data structures that are (potentially) infinite. In this section, we will give a small example of how this can be done. We cannot give the example in ML because it uses a by-value strategy. The example is possible, though, in Haskell. So, to avoid loading the reader with a different syntax, we will write the terms with the same concrete syntax as ML but with the stipulation that evaluation must be understood as lazy.

First, we have to clarify the concept of value for data structures such as lists. In a language that uses eager evaluation, a value of type `T list` is a list whose elements are *values* of type `T`. In a lazy language, this is not a good notion of value because it might require the evaluation of useless redexes, contrary to the call-by-need philosophy. To see the reason, define the functions:

```
fun hd x::rest = x;
fun tl x::rest = rest;
```

The functions return, respectively, the first element (that is, the head) and the rest (that is, the tail) of a non-empty list (in the case of the empty list, the abstract machine will generate an error when doing pattern matching).

Let us now consider the expression

```
hd [2, ((fn n=>n+1) 2)].
```

To calculate its value (that is, 2), it is not necessary to reduce the redex that comprises the second element of the list because it will never be used in the body of the `hd` function. For these reasons, in a by-need context, a value of type list is any expression of the form:

```
exp1 :: exp2
```

where `exp1` and `exp2` can also contain redexes.

It is then possible to define a list by recursion, as for example:

```
val infinity2 = 2 :: infinity2;
```

The expression `infinity2` corresponds to a potentially infinite list whose elements are all 2 (using eager evaluation, such an expression would cause a divergent computation). We may manipulate `infinity2` under lazy evaluation. For example,

```
hd infinity2
```

is an expression whose evaluation terminates with the value 2. Another expression with value 2 is

```
hd (tl (tl (tl infinity2))).
```

Side Effects in LISP

LISP, the first functional language, is based on a data structure called a *dotted pair* (or cons cell, see the box on Sect. 6.1.1). Dotted pairs are composed of two parts: the *car* (for *contents of the address register*) and *cdr* (for *contents of the decrement register*). An expression of the form:

```
(cons a b)
```

allocates a dotted pair and initialises it so that its *car* points to the value of *a* and its *cdr* points to the value of *b*. The two components of a dotted pair can be selected using the functions *car* and *cdr*. So,

```
(cdr (cons a b))
```

has the value *b*. These are characteristics that belong to the pure part of LISP.

LISP has, in addition, imperative mechanisms. For example, there are functions such as (*rplaca* *x a*) and (*rplacd* *x a*) which, respectively, *assign to the car* and *assign to the cdr* of the dotted pair *x* the value of *a*:

```
(cdr (rplacd (cons 'a 'b) 'c))
```

This has the value *'c*. Other side-effecting functions are *set* and *setq*.

As a last example of a stream, the following function constructs an infinite list of natural numbers starting with its argument, *n*:

```
fun numbersFrom n = n :: numbersFrom(n+1);
```

We can define a higher-order function that applies its functional argument to all the elements of a list, as in:

```
fun map f nil = nil
  | map f e::rest = f(e)::rest;
```

We can now produce the infinite list of all the squares starting from *n***n*:

```
fun squaresFrom n = map (fn y => y*y) (numbersFrom n);
```

11.3.7 Imperative Aspects

Many functional languages also include imperative mechanisms which introduce a notion of state that can be modified by side effects.

In ML, there are real modifiable variables (called “reference cells”) with their own types. For every type *T* in the language (including functional types, therefore), the type *T ref* is defined and its values are modifiable variables that can contain values of type *T*. A modifiable variable of type *T*, initialised to the value *v* (of type *T*) is created by:

```
ref v
```

The usual constructs that associate names and values can also be used on values of type *T ref*. For example:

```
val I = ref 4;
```

This creates a reference cell of type `int ref`, which is initialised to 4 and has the name `I`. This is what, in imperative languages, we call “the modifiable variable (with name) `I`.” Obviously, `I` is the name of the reference cell and not of the value it contains (it is an l-value). To obtain its r-value, it is necessary to dereference it explicitly using the `!` operator:

```
val n = !I + 1;
```

The expression `!I` is of type `int`. In general, if `V` is an expression of type `T ref`, `!V` is of type `T`. The line above associates the name `n` with the value 5. At risk of being pedantic, let us observe that `n` is not a modifiable variable, it is an ordinary name bound to a value in the environment.

Reference cells can be modified using assignments:

```
I := !I + 1;
```

Note the difference with the definition of the value `n` given above. Here, we are modifying (using a side effect) the r-value of a reference cell that already exists. The type of an imperative construct of this kind is `unit` (recall Sect. 8.3.7).

The presence of modifiable variables clearly introduces the possibility of aliasing. Let us indeed consider the following definitions:

```
val I = ref 4;
val J = I;
```

The second line associates the value of the name `I` (that is, the reference cell—the l-value) with the name `J`. We here have a classic situation of aliasing: `I` and `J` are different names for the same l-value. After execution of the fragment

```
I := 5;
val z = !J;
```

the name `z` is associated with the value 5.

The language definition does not specify how to implement an assignment between modifiable variables. From the examples given so far using integers, one could imagine a traditional implementation with the r-value being copied from the source to the destination of the assignment. However, values with quite different memory sizes can be stored in reference cells. Let us consider, for example, modifiable variables that contain lists and strings:

```
val S = ref "pear";
val L = ref ["one", "two", "three"];
```

The name `S` is of type `string ref`, while `L` is of type `(string list) ref`. The two modifiable variables associated with the names `S` and `L` can contain, respectively, *any* string and *any* list. For example, we can perform two assignments:

```
S := "this is a string much longer than previously";
L := "zero" :: "four" :: "five" :: !L;
```

The new values of L and S require more memory than their previous values did. It is not possible to implement these assignments using a simple (and traditional) value copy. The abstract machine will, in this case, copy references (or pointers) to these values. All of this is, though, completely invisible to the language user. The implementation handles the two space requirements by allocating the necessary memory for each case and then modifies the references.

As well as modifiable variables, ML also provides imperative control constructs, such as sequential composition ($;$) and loops.

It is appropriate to note that, in the presence of these imperative features, both Theorem 11.1 and the property that we cited immediately after it *do not apply*. This is particularly relevant in the case of a lazy evaluation strategy which would become incorrect in the presence of side effects. It is for this reason that languages such as Miranda and Haskell, which use lazy evaluation, do not admit side effects of any kind (they are purely functional languages).

11.4 An Assessment

The computing beginner often holds that design and writing efficient programs are the most important tasks on which their occupation is based. Software Engineering, however, has largely shown, both in theory, as in ample experimental studies, that the most critical factors in a software project are its correctness, readability, maintainability and its dependability. In economic terms, these factors account for more than fifty percent of the total cost; in social terms, software maintenance (which depends in a critical way on its readability) can involve hundreds of different people over a period of tens of years. In ethical terms, the life, or health, of thousands of people can depend on the reliability and correctness of a software system.

The goal of the research in program correctness (which uses tools and techniques from many different areas of computer science) is to produce software with correctness guarantees comparable to those with which structural engineers release their own products (bridges, columns, structures). They have at their disposal a whole corpus of applied mathematics with which to “calculate” the structures. If a bridge must take a certain load, be subjected to certain winds, can be exposed to seismic events of a given magnitude, etc., it must have given dimensions, must be built using materials with certain characteristics, etc. These characteristics are not determined by the designer’s taste but are calculated using appropriate techniques.

Reasoning about programs with side effects is particularly difficult and expensive. On the contrary, there are standard techniques that allow reasoning about side-effect-free programs. If reliability, readability, and correctness are more important than efficiency, there is no doubt that functional programming generates more readable software, whose correctness is easier to establish and, therefore, is more reliable. Indeed, a function may be seen as a behaviour, and several simple behaviours can be combined to obtain more complex ones.

This is true also for non-pure functional programming languages—and for conventional imperative languages with a significant functional programming subset—where we may isolate significant software components which are indeed stateless,

and thus one may reason on those components in isolation, with the guarantee that they will always behave in the same manner, since no side-effect is around.¹² It is for these reasons that higher-order functions as first-class citizens are one of the major components of any modern programming language, of any paradigm. Mechanisms for concurrent programming (for instance, futures or callbacks) are most of the times based on functions. Designing and implementing a concurrent system in terms of value transformations (“term rewriting”) instead of state transformations, makes it easier to deal with the inherent non-determinism of these systems.

Let us stress, finally, that functional languages had a tremendous impact on the *design* of programming languages of any paradigm. Many concepts and experimental features in functional programming have later migrated to other paradigms. Among these concepts, type systems, generics, polymorphism, type safety, all originated in functional languages (because is it simpler to study and to implement them in an environment without side effects).

11.5 Implementation: The SECD Machine

The technology for implementing functional programming languages is today sophisticated and cannot be adequately described in this book. The techniques for passing functions as parameters we described in Chap. 7 constitute the heart of these implementations. In order to give more details on how to handle higher-order functions, we will limit ourselves to summarising the SECD machine, a prototype proposed in 1964 by Peter Landin for evaluation *by value*. Many other abstract machines have developed out of the SECD machine.

We will describe the SECD machine in an abstract manner, using a very simple, purely functional language that uses only abstraction and application. The primitive elements of the language are the names of constants and variables and some primitive functions (for example, we could have constants `0`, `1`, `true`, etc. and primitive functions `succ` and `pred`). The important part of the machine is not concerned with these aspects of the language but with the presence of higher-order functions. Let us assume, for simplicity, that we are only dealing with unary functions. Let us assume that *Var* is a non-terminal from which can be derived a denumerable number of names (of variables) and that *Const* and *Fun* are, respectively, the non-terminals from which are derived the appropriate constants and primitive functions. We can state the grammar of our example language as:

¹² One major example is the Erlang programming language, designed at Ericsson for building scalable, concurrent systems with high availability guarantees; or, in more concrete terms, “to provide a better way of programming telephony applications” [1]. An Erlang application is a collection of many, small processes that communicate through messages. Any such lightweight process is purely functional (although the system resulting from their composition is not). In 1998 Ericsson announced the AXD301 switch, with more than one million of lines of Erlang code, probably the largest ever program in a functional language, at that time. The AXD301 was extremely reliable, and this was related to the quality of the functional design.

$$exp:: = Const \mid Var \mid Fun \mid (exp \ exp) \mid (\text{fn } Var \Rightarrow exp).$$

The SECD machine has four main components:

- A **Stack** to store the partial results which will be used during the computation.
- An **Environment**, that is, a list of associations between names and values. Among values, there are abstractions (since “there is no evaluation under an abstraction”). To deal with abstractions with free variables we need to couple the abstraction to the environment in which the free variables in its body are to be evaluated (a concept we called a *closure* in Chap. 7). Let us assume that a closure is a *triple* composed of an environment, an expression (which will always be the body of an abstraction) and a variable (representing the variable that was bound in the abstraction). We will therefore write a closure as $cl(E, exp, x)$, which represents the abstraction $(\text{fn } x \Rightarrow exp)$ in the environment E .
- A **Control**, represented by a stack of expressions to be evaluated. Among the expressions, we have the special operator $@$, which is read “app” and which indicates that an application should be executed (because the two expressions that comprise an application have already been evaluated and are stored in the next two positions on the stack).
- A storage area called the **Dump**, that is a stack in which previous states of the machine have been saved when the computation was suspended to evaluate internal redexes.

We write $[a_1, \dots, a_n]$ to denote the stack composed of the elements $a_1, \dots, a_n$ with a_1 at its top. $[]$ is the empty stack. If P is a stack, $top(P)$ denotes the topmost element, while $tl(P)$ denotes P with the top element removed.

A state of the machine is composed of a quadruple, (S, E, C, D) . A dump is always composed of a structure of the form:

$$(S_1, E_1, C_1, (S_2, E_2, C_2, (S_3, E_3, C_3, []))).$$

This state of the dump has been obtained by first saving the state

$$(S_3, E_3, C_3, [])$$

then the state

$$(S_2, E_2, C_2, (S_3, E_3, C_3, []))$$

which includes the previous dump, and so on.

To evaluate an expression, exp , the machine starts its operation in the state $([], [], [exp], [])$ and terminates when both the control, C , and the dump, D , are empty.

The operation of the SECD is described by a transition function which allows it to pass from one state to a succeeding one using the following rules. The machine selects the rule according to the expression on the control list (imagine that control is described using abstract syntax). Let us assume, therefore, that the machine finds itself in the generic state, (S, E, C, D) , then:

1. A constant, c (e.g., 1, 0, *true*), is on top of the control list, that is, there is an expression that does not need to be further evaluated. In this case, the new state of the machine is $(c :: S, E, tl(C), D)$ in which the (immediate) value of the expression is pushed onto the stack and the expression just evaluated is removed from the control list.
2. There is a variable, x , on the top of the control list. In this case, the value is the one which the environment associates with x . Therefore, the new state of the machine is $(E(x) :: S, E, tl(C), D)$.
3. On the top of the control list, there is an application of the form $(exp_f\ exp_a)$. In this case, since the SECD implements eager evaluation, it is necessary to follow the evaluation procedure we described in Sect. 11.2.3. The new state is therefore $(S, E, exp_f :: exp_a :: @ :: tl(C), D)$ which expresses the fact that the next expression to be evaluated is the functional part of the application (which at some point will have as its value a closure, as we will see), followed by the argument part of the application, followed, finally, by the special expression, $@$, which will force the actual application of the function to its argument.
4. On the top of the control list, there is an abstraction, $(\lambda x.\ exp)$. We know that we are already in the presence of a value which must however be “closed” with the current environment. The new state of the machine is therefore $(cl(E, exp, x) :: S, E, tl(C), D)$.
5. The special value $@$ is on the top of the control list. We then know, by construction, that on the stack there are two values that represent, respectively, an argument (on the top of the stack) and a function to apply (the second element). There are two subcases which depend on the function to be applied:
 - The function is a *primitive function*, f , such as *succ*. The function must be directly applied to its argument and the new state of the machine is $(f(top(S)) :: tl(tl(S)), E, tl(C), D)$.
 - The function on the top is a closure of the form $cl(E_1, exp, x)$. This is one of the central points of the SECD machine; it brings the dump into play. It is necessary to use the copy rule. The computation passes to exp in the environment of the closure, E_1 , (and not in the current environment, E) modified by the binding of the formal parameter. But the computation performed up to now must not be forgotten. It is frozen on the dump. The new state of the machine is therefore $([], E_1[x \leftarrow top(S)], [exp], (tl(tl(S)), E, tl(C), D))$.
6. The control list is empty. This can happen when the computation has terminated (in which case, the dump is also empty). It can also happen when a computation started by the previous rule (the one for applying a closure to a value) is finished. In this second case (that is, non-empty dump of the form (S_1, E_1, C_1, D_1)), it is necessary to resume the computation saved on the dump, restoring, at the same time, the value just computed (which is on the top of the stack). The new state of the machine is $(top(S) :: S_1, E_1, C_1, D_1)$.

S	E	C	D r
[]	[]	$[(F\ 3)]$	[]
[]	[]	$[F, 3, @]$	[] 3
$[ch_1]$	[]	$[3, @]$	[] 4
	where $ch_1 = cl([], (sqrt\ ((\text{fn } y \Rightarrow (succ\ x))\ x)), x)$		
$[3, ch_1]$	[]	$[@]$	[] 1
[]	$[x \leftarrow 3]$	$[(sqrt\ ((\text{fn } y \Rightarrow (succ\ x))\ x))]$	$d_1\ 5$
	where $d_1 = ([], [], [], [])$		
[]	$[x \leftarrow 3]$	$[sqrt, ((\text{fn } y \Rightarrow (succ\ x))\ x), @]$	$d_1\ 3$
$[sqrt]$	$[x \leftarrow 3]$	$[(\text{fn } y \Rightarrow (succ\ x))\ x], @]$	$d_1\ 1$
$[sqrt]$	$[x \leftarrow 3]$	$[(\text{fn } y \Rightarrow (succ\ x)), x, @, @]$	$d_1\ 3$
$[ch_2, sqrt]$	$[x \leftarrow 3]$	$[x, @, @]$	$d_1\ 4$
	where $ch_2 = cl([x \leftarrow 3], (succ\ x), y)$		
$[3, ch_2, sqrt]$	$[x \leftarrow 3]$	$[@, @]$	$d_1\ 2$
[]	$[x \leftarrow 3, y \leftarrow 3]$	$[(succ\ x)]$	$d_2\ 5$
	where $d_2 = ([sqrt], [x \leftarrow 3], [@], d_1)$		
[]	$[x \leftarrow 3, y \leftarrow 3]$	$[succ, x, @]$	$d_2\ 3$
$[succ]$	$[x \leftarrow 3, y \leftarrow 3]$	$[x, @]$	$d_2\ 1$
$[3, succ]$	$[x \leftarrow 3, y \leftarrow 3]$	$[@]$	$d_2\ 2$
$[4]$	$[x \leftarrow 3, y \leftarrow 3]$	[]	$d_2\ 5$
$[4, sqrt]$	$[x \leftarrow 3]$	$[@]$	$d_1\ 6$
$[2]$	$[x \leftarrow 3]$	[]	$d_1\ 5$
$[2]$	[]	[]	[] 6

Fig. 11.3 A computation using the SECD machine

Figure 11.3 shows how the SECD machine computes the value of the expression $(F\ 3)$, where:

$$F = \text{fn } x \Rightarrow (sqrt\ ((\text{fn } y \Rightarrow (succ\ x))\ x)).$$

The computation is shown as a table in which every row represents a transition of the machine. The rule being used for the transition is indicated in the rightmost column.

11.6 Fundamentals: The λ -Calculus

At the start of the chapter, we saw that the functional paradigm is inspired by a foundation for computability theory, different from that based on Turing machines (but equivalent to that as far as expressive power is concerned). We present in this section some of the ingredients of this formal system for computability— λ -calculus.

The syntax of the λ -calculus is extremely austere and can be given in one line using the following grammar, where X is a non-terminal which represents a generic variable, M is the initial symbol, the dot and the two parentheses are terminal symbols:

$$M ::= X \mid (MM) \mid (\lambda X.M).$$

In addition, let us assume that we also have a denumerable set of terminal symbols for variables which, for ease of notation, we will in general write using the last lower-case letters of the alphabet: x, y, z , etc.

The reader will recognise in these clauses both application and abstraction (written using λ instead of fn) which we introduced informally in Sect. 11.1.3. We call λ -terms (or, simply, terms) the strings of terminals that are derived from this grammar.

Syntactic Conventions, Free and Bound Variables

The “official” books on the λ -calculus introduce many additional conventions to simplify the notation. For example, application associates to the left, or:

$$M_1 M_2 \cdots M_n \text{ stand for } (\cdots (M_1 M_2) \cdots M_n).$$

The scope of a λ extends as far as possible to the right (that is, $\lambda x.x y$ stands for $(\lambda x.(x y))$ and not for $((\lambda x.x).y)$). We will seek to use these conventions as little as possible by using parentheses where necessary. We will also speak of *subterms*, with the obvious meaning (for example, $(x y)$ is a subterm of $(\lambda x.(x y))$ but λx is not).

The abstraction operator *binds* the variable upon which it acts,¹³ in the double sense that, semantically, consistent renaming of the bound variable does not modify the semantics of an expression and, syntactically, possible substitutions have no effect on bound variables. We formalise these aspects in the following definitions. First, we define, for an arbitrary expression, M , the set of its *free variables*, which we denote by $Fv(M)$, and of its bound variables $Bv(M)$:

$$\begin{aligned} Fv(x) &= \{x\} & Bv(x) &= \emptyset, \\ Fv(MN) &= Fv(M) \cup Fv(N) & Bv(MN) &= Bv(M) \cup Bv(N), \\ Fv(\lambda x.M) &= Fv(M) - \{x\} & Bv(\lambda x.M) &= Bv(M) \cup \{x\}. \end{aligned}$$

Substitution

We can formally define the concept of substitution without variable capture which constitutes the core of the copy rule. We define, therefore, the notation, $M[N/x]$ which we read as the substitution of N for the free occurrences of x in M :

$$\begin{aligned} x[N/x] &= N, \\ y[N/x] &= y & \text{when } x \neq y, \\ (M_1 M_2)[N/x] &= (M_1[N/x] M_2[N/x]), \\ (\lambda y.M)[N/x] &= (\lambda y.M[N/x]) & \text{when } x \neq y \text{ e } y \notin Fv(N), \\ (\lambda y.M)[N/x] &= (\lambda y.M) & \text{when } x = y. \end{aligned}$$

The definition is simple. Practically, to substitute N in place of x in M , we “push” the substitution down towards the leaves of the syntax tree, that is to the variables, and check if the leaf is labelled with x or with some other variable. But the definition is subtle, since it avoids capture of the variables in N by some λ . This is the reason

¹³ Remember what was said in Sect. 7.1 about formal parameters to procedures.

why the last clause requires that $y \notin Fv(N)$. What happens if, on the other hand, y is present in N ? We can rename the variable bound by λ . As we said above, indeed, it is not the name of the variable that counts when a λ is applied, but only the way in which it is used. In other words, we will consider as equivalent two terms that differ only by the names of their bound variables. For example, $\lambda x.x$ and $\lambda y.y$, or $\lambda x.\lambda y.x$ and $\lambda v.\lambda w.v$.

Alpha Equivalence

The intuitive idea that two expressions are equivalent when they differ only in the names of their free variables is formalised in the concept of α -equivalence:

$$\lambda x.M \equiv_{\alpha} \lambda y.M[y/x] \quad y \text{ fresh}$$

where by “ y fresh”, we mean that y is a new variable not present in M .

Two terms that differ by just the replacement of some subterm of one term by an α -equivalent subterm will be considered identical.

Computation: β Reduction

Computation proceeds by rewriting according to the prescriptions of β -reduction:

$$(\lambda x.M)N \rightarrow_{\beta} M[N/x].$$

More generally, we will say that M β -reduces to N (and we write $M \rightarrow N$) when N is the result of the application of one step of β -reduction to some subterm of M . A subterm of the form $(\lambda x.M)N$ is a *redex* whose *reductum* is $M[N/x]$. Note that the concept of β -reduction is not deterministic. Whenever there exists more than one redex in the same term, it is not prescribed which one will be chosen. Observe that the notion of reduction that we just defined does not restrict the position of the reduced redex. It may well be the case that the redex is contained in the body of an abstraction. Contrary to the reduction we defined in Sect. 11.2.1, here we do allow reduction under abstraction.

β -reduction is a relation that is not symmetric. In general, if $M \rightarrow N$, it is not the case that $N \rightarrow M$. It is useful to introduce a symmetric relation which we call β -equivalence, defined as the reflexive and transitive closure of β -reduction and denoted by $=_{\beta}$. Intuitively, $M =_{\beta} N$ means that M and N are connected through a sequence of β -reductions (not necessarily all in the same direction).

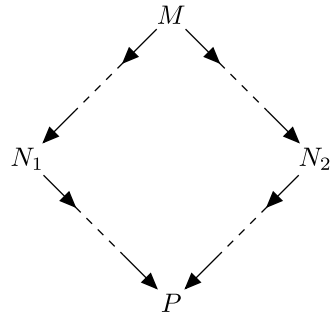
Normal Forms

When a λ -term that does not contain a redex, β -reduction terminates; such terms are called *normal forms*. For example, $\lambda x.\lambda y.x$ is a normal form, while the term $\lambda x.(\lambda y.y)x$ is not a normal form because it contains the redex $(\lambda y.y)x$. We have, then:

$$\lambda x.(\lambda y.y)x \rightarrow \lambda x.x$$

and $\lambda x.x$ is now a normal form.

Fig. 11.4 The confluence property for λ -terms



Some terms have reductions that terminate (in a normal form). There are however also terms which reduce an infinite number of times without ever producing a normal form, as for example:

$$\begin{aligned}
 (\lambda x.xx)(\lambda x.xx) &\rightarrow (xx)[(\lambda x.xx)/x] \\
 &= (\lambda x.xx)(\lambda x.xx) \\
 &\rightarrow (\lambda x.xx)(\lambda x.xx) \\
 &\rightarrow \dots
 \end{aligned}$$

Confluence

A fundamental property of the λ -calculus is that its inherent non-determinism has no dangerous effects. In any way we might choose the redex to reduce inside a term, the final result (the normal form) of a sequence of reductions is always the same. Stated in a more formal manner, we have the following property, called *confluence* and graphically represented in Fig. 11.4.

If M reduces to N_1 in a number of reduction steps, and M also reduces to N_2 in a number of reduction steps, then there exists a term, P , such that both N_1 and N_2 both reduce to P in a number of steps.

An important consequence of this property is that if a term can be reduced to a normal form, this normal form is unique and independent of the path followed to reach it. As we already observed on Sect. 11.2.4, crucial for this property to hold in full generality (and not only in the restricted sense of Theorem 11.1) is the possibility of reducing under an abstraction.

Fixed Point Operators

We have seen how the λ -calculus formalises many concepts that we have seen in this chapter.¹⁴ We have not however introduced any mechanisms for the definition of

¹⁴ We should rather say: formalises many of the concepts that we have seen in this *book*. The λ -calculus was a fundamental inspiration to the design of programming languages starting with

recursive functions. The λ -calculus has no notion of an external or global environment in which to assign names to terms, as it happens when defining a recursive function, for which naming seems to be a fundamental concept.

The point (surprising at first sight) is that the concepts of abstraction and local environment (which is derived from the possibility of nested λ 's) are *on their own sufficient* for the definition of recursion or, more precisely, fixed points.

Let us consider the following very simple recursive definition, which, for simplicity, we write in a λ -calculus extended with integers and conditional expressions¹⁵:

$$Z = \lambda n. \text{ if } n = 0 \text{ then } 0 \text{ else } 1 + Z(n - 1). \quad (11.1)$$

Our experience with programming languages suggests, perhaps, that we should read this relation as a *definition*:

$$Z \stackrel{\text{def}}{=} \lambda n. \text{ if } n = 0 \text{ then } 0 \text{ else } 1 + Z(n - 1).$$

But this is not the only reading possible (in particular, it is not possible in the λ -calculus, which has no concept of global environment to allow us to use the name of a term inside an expression). A mathematician would probably read it as an *equation* with unknown Z (recall the discussion on inductive definitions in Sect. 6.5). For this very simple equation, it is clear that one solution (rather, in this case, the unique solution) is the identity function:

$$\mathcal{I}(n) = n. \quad (11.2)$$

What we are looking for is a general method which allows us to pass from an equation such as (11.1) to an *algorithm*, that is to a λ -term, which is a solution to this equation when the equality symbol is interpreted as β -equivalence:

$$Z =_{\beta} \lambda n. \text{ if } n = 0 \text{ then } 0 \text{ else } 1 + Z(n - 1). \quad (11.3)$$

Note that this is to ask more than simply provide an arbitrary function such as (11.2). It requires a term which *in the calculus* satisfies (11.3).¹⁶

Let us consider, first, the function that is obtained from (11.3) by λ -abstracting on the unknown function:

$$F \stackrel{\text{def}}{=} \lambda f. \lambda n. \text{ if } n = 0 \text{ then } 0 \text{ else } 1 + f(n - 1).$$

Hence our problem is to find a suitable term g such that:

$$g =_{\beta} F(g).$$

ALGOL and LISP. Concepts such as scope, local environment, call by name, to mention only the most important, have been “imported” into programming languages on the basis of analogous concepts in the λ -calculus.

¹⁵ We use this extension only to simplify the presentation. In the λ -calculus, it is possible to *codify* numbers and conditions using only variables, abstractions and applications.

¹⁶ The term $\lambda n. n$ is an algorithm for the only function which satisfies (11.1), but it is *not* a solution of (11.3), which expresses a determinate behaviour through reduction.

A term, g , which satisfies an equation of this type is called a *fixed point* of F . Therefore our problem of finding a solution to (11.3) can be reformulated as a problem of finding the fixed point of some function.

We want now to show that the λ -calculus provides a general method to “automatically” solve these equations. Let us, indeed, consider the following term, which for convenience (and following tradition), we name Y :

$$Y \stackrel{\text{def}}{=} \lambda f. (\lambda x. f(x x)) (\lambda x. f(x x)).$$

This is a term which can be reduced an infinite number of times without reaching normal form. But it can be put to good use. Let us indeed try to apply it to an arbitrary term, M :

$$\begin{aligned} YM &=_{\beta} (\lambda x. M(x x)) (\lambda x. M(x x)) \\ &=_{\beta} M((\lambda x. M(x x)) (\lambda x. M(x x))) \\ &=_{\beta} M(YM). \end{aligned}$$

In the last step, we have used the fact that was established in the first line, that

$$YM =_{\beta} (\lambda x. M(x x)) (\lambda x. M(x x))$$

and we have then substituted equals for equals. YM is therefore a fixed point of M : Y is a term in the calculus which can compute the fixed point of an arbitrary term.¹⁷

We say that Y is a *fixed point operator*. It is then quite clear that Y provides solutions to any recursive equation. In particular, it gives us a solution to (11.3):

$$Id \stackrel{\text{def}}{=} YF.$$

The term Id thus defined has a reduction behaviour which satisfies the relation with which we started. It is instructive to try to apply this function to a specific argument to see how the mechanism of fixed points is really capable of computing a recursive function. For example:

$$\begin{aligned} Id\ 2 &\stackrel{\text{def}}{=} (YF)\ 2 \\ &\rightarrow F(YF)\ 2 \\ &\rightarrow \text{if } 2 = 0 \text{ then } 0 \text{ else } 1 + ((YF)(2 - 1)) \\ &\rightarrow 1 + ((YF)\ 1) \\ &\rightarrow 1 + (F(YF)\ 1) \\ &\rightarrow 1 + ((\text{if } 1 = 0 \text{ then } 0 \text{ else } 1 + ((YF)(1 - 1)))) \\ &\rightarrow 1 + (1 + ((YF)\ 0)) \\ &\rightarrow 1 + (1 + (\text{if } 0 = 0 \text{ then } 0 \text{ else } 1 + ((YF)\ (0 - 1)))) \\ &\rightarrow 1 + (1 + 0) \\ &\rightarrow 2. \end{aligned}$$

¹⁷ This shows also that, in the λ -calculus, every term has always at least one fixed point, that computed by Y .

At each “recursive call”, YF provides a new copy of the body of the function which is used on the next call.

What we have just seen is also true for the definition of recursive values which are not functions. The stream that we defined in Sect. 11.3.6 as:

$$\text{val infinity2} = 2 :: \text{infinity2};$$

is nothing more than a fixed point:

$$\text{infinity2} \stackrel{\text{def}}{=} Y(\lambda l. 2 :: l).$$

Observe that Y is a fixed point operator with respect to β -equality.¹⁸ If, instead of β -reduction, we use a different strategy, for example, the analogue of reduction by value which we defined for functional languages, Y is no longer able to compute fixed points. For reduction by value, another operator must be used, for example:

$$H \stackrel{\text{def}}{=} \lambda g. ((\lambda f. ff)(\lambda f. (g(\lambda x. ffx)))).$$

For reasons of efficiency, implementations of functional programming languages do not use fixed point operators to implement recursion. However, fixed point operators play a role that is very important to the theory of these languages.

Expressiveness of the λ -Calculus

This very simple formal system, constructed only from the concepts of application and abstraction, is not as rudimentary as it seems, if it can express arbitrary recursions. Indeed, it is possible to show that the λ -calculus is a Turing-complete formalism. In the first place, it is possible to encode the natural numbers as λ -terms (a λ -term, $\underline{n}$ is associated with the number n) and, then, show that to every computable function, f , there corresponds a λ -term M_f which computes f : if $f(n) = m$, then $M_f \underline{n} \rightarrow \underline{m}$.

11.7 Summary

In this chapter, we have presented the functional programming paradigm. In its *pure* form, this is a computational model which does not contain the concept of modifiable variable. Computation proceeds by the rewriting of terms that denote functions. The main concepts that we have discussed are:

- *Abstraction*. A mechanism for passing from one expression denoting a value to one denoting a function.
- *Application*. A mechanism dual to abstraction, by which a function is applied to an argument.

¹⁸ Clearly it is not unique. For another operator slightly more complex than Y but which satisfies a stronger relation, see Exercise 7.

- *Computation by rewriting* or *reduction*, in which an expression is repeatedly simplified until a form that can not be further reduced is encountered.
- *Redex*. The syntactic structure which is simplified during reduction and is formed from the application of an abstraction to an expression.
- The centrality of *higher order* in this mode of computation.
- The concept of *value* which corresponds to those syntactic forms not otherwise reducible and where reduction therefore terminates.
- The different *evaluation strategies*: by value (or eager), by name, lazy.
- Some mechanisms that the most common functional languages add to the fundamental nucleus. Among these, recall: an interactive environment, a rich system of types, pattern matching, imperative aspects such as variables and assignments.
- A comparison of the functional and imperative paradigms, centred on the notion of program correctness.
- The *SECD machine*, a prototype abstract machine for higher-order functional languages using the by-value strategy.
- The λ -calculus, a simple and powerful formal system that constitutes the formal nucleus of all functional languages.

11.8 Bibliographical Note

Reading the original article by John McCarthy on LISP is still a valuable experience [2]. An elementary introduction to programming using ML is to be found in [3], while [4] is a more advanced text (which uses the Caml dialect); [5] is the official definition of the language and it is useful to read it together with the commentary [6].

The implementation of functional languages, with particular regard to lazy evaluation is dealt with in [7]. The SECD machine was introduced in the pioneering work of Peter Landin [8].

John Backus' lecture for the Turing Award [9] should be essential reading (at least for its first part) for every student of programming languages.

The lambda calculus was introduced in the 1930s by Alonzo Church. An original reference is [10]. The modern reference text is [11], even if [12] is a more accessible introduction.

11.9 Exercises

1. Consider the following definitions in ML:

```
fun K x y = x;  
fun I x = x;  
fun Omega x = Omega x;
```

State what the result of evaluating the following expression will be.

$K \ (I \ 3) \ (\Omega\omega \ 1) \ .$

What would be the result if ML adopted a lazy strategy?

2. Write a function `map` which applies its first parameter (which will be a function) to all the elements of a list passed as its second argument.
3. Describe formally the operation of the SECD machine, by giving a definition using the structural operational semantics that we introduced in Sect. 2.5.
4. In λ -calculus, we define the code of a natural number, n , as:

$$\underline{n} \stackrel{def}{=} \lambda f. \lambda x. f^n x \stackrel{def}{=} \lambda f. \lambda x. f(f \cdots (fx) \cdots),$$

where, in the body, there are n occurrences of f . Give the definition of a term, *prod* such that:

$$prod \ \underline{n} \ \underline{m} \rightarrow \underline{n * m}.$$

5. Apply the technique of calculating fixed points to solve the following equation for the unknown *fact*:

$$fact =_{\beta} \lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * fact(n - 1)$$

and describe the reduction which computes the value of its solution applied to argument 2.

6. Apply the technique of calculating fixed points to solve the following equation for the unknown *h*:

$$h =_{\beta} \lambda x. \text{if } x > 100 \text{ then } x - 10 \text{ else } h(h(x + 11)).$$

Call the solution *Ninetyone*, then calculate *Ninetyone*(99).

7. Consider the λ -terms:

$$A = \lambda x. \lambda y. y(xxy),$$

$$\Theta = AA.$$

Show that Θ is a fixed-point operator for which the relation:

$$\Theta M \rightarrow \cdots \rightarrow M \ (\Theta M).$$

is true. This relation is stronger than that satisfied by Y , for which only $YM =_{\beta} M(YM)$ is true. The term, Θ , is called the Turing fixed point operator.

References

1. J. Armstrong, A history of Erlang, in *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages*, HOPL III (Association for Computing Machinery, New York, 2007), pp. 6.1–6.26
2. J. McCarthy, Recursive functions of symbolic expressions and their computation by machine, part I. *Commun. ACM* **3**(4), 184–195 (1960)
3. J.D. Ullman, *Elements of ML Programming* (Prentice-Hall Inc, Upper Saddle River, 1994)
4. G. Cousineau, M. Mauny, *The Functional Approach to Programming* (Cambridge University Press, 1998)

5. R. Milner, M. Tofte, R. Harper, D. MacQueen, *The Definition of Standard ML - Revised* (MIT Press, 1997)
6. R. Milner, M. Tofte, *Commentary on Standard ML* (MIT Press, 1991)
7. S. Peyton-Jones, *The Implementation of Functional Programming Languages* (Prentice Hall, 1987)
8. P.J. Landin, The mechanical evaluation of expressions. *Comput. J.* **6**(4), 308–320 (1964)
9. J. Backus, Can programming be liberated from the von Neumann style? A functional style and its algebra of programs. *Commun. ACM* **21**(8), 613–641 (1978). Turing Award Lecture
10. A. Church, *The Calculi of Lambda Conversion* (Princeton University Press, 1941)
11. H. Barendregt, *The Lambda Calculus: Its Syntax and Semantics* (Elsevier, 1984)
12. R. Hindley, P. Seldin, *Introduction to Combinators and Lambda-Calculus* (Cambridge University Press, 1986)

12.1 Deduction as Computation

A famous slogan by R. Kowalski, accurately captures the concepts underpinning the activity of programming: $\text{Algorithm} = \text{Logic} + \text{Control}$. According to this “equation”, the specification of an algorithm, and therefore its formulation in programming languages, can be separated into two parts. On the one side, the logic of the solution is specified—that is, the “what” must be done is defined. On the other, those aspects related to control are specified, and therefore the “how” of finding the desired solution is clarified. The programmer who uses a traditional imperative language must take account of both these components. Logic programming, on the other hand, was originally defined with the idea of separating these two aspects. The programmer is only required, at least in principle, to provide a logical specification. Everything related to control is delegated to the abstract machine. Using a computational mechanism based on a particular deduction rule (resolution), the interpreter searches through the space of possible solutions for the one specified by the “logic”, defining in this way the sequence of operations necessary to reach the final result.

The basis for this view of computation as logical deduction can be traced back to the work of K. Gödel and J. Herbrand in the 1930s. In particular, Herbrand anticipated, albeit incompletely, some ideas about the process of unification, which, as we shall see, is the basic computational mechanism of logic programming languages.

It was not until the 1960s that a formal definition (due to A. Robinson) of this process appeared. Then, ten years later (in the early 1970s) it was realised that formal automatic deduction of a particular kind could be interpreted as a computational mechanism. The first programming languages of this paradigm were created, among which PROLOG (the name is an acronym for PROgramming in LOGic; for more information on history, see Sect. 16.4).

Today there are many implemented versions of PROLOG, and there are several other languages in this paradigm (as far as applications are concerned, those including constraints are of particular interest). All these languages (including PROLOG) also

provide constructs for the explicit specification of control, for the sake of efficiency. Since these constructs do not have a direct logical interpretation, they make the semantics of the language more complicated and cause the loss of part of the purely declarative nature of the logic paradigm. This notwithstanding, we still call them logic programming languages—they require the programmer to do little more than formulate (or declare) the specification of the problem to be solved. In some cases, the resulting programs are really surprising in their brevity, simplicity and clarity, as we will see in the next section.

Terminological Note

We should distinguish between Logic programming, which is the theoretical foundational formalism, from PROLOG, a language that has different implementations. The concepts we introduce below, if not otherwise stated, are valid for both. The important differences will be explicitly pointed out. The programming examples which we include (usually in `this` font) are all PROLOG code that could be run on some implementation of the language. The theoretical concepts, on the other hand, even when they are valid also for PROLOG, use mathematical concepts and are rendered in *italic*. Moreover, we will follow the PROLOG convention that we always write variables beginning with an upper-case letter. As mentioned before, in the logic programming paradigm there exist several other languages, different from PROLOG. We call all of them logic programming languages, or logic languages, for short.

12.1.1 An Example

We start with a fairly informal example since we did not yet introduce either the syntax or the semantics of logic languages.

Let us consider the following problem. We want to arrange three 1s, three 2s, . . . , three 9s in a list (which therefore will consist of 27 numbers) such that, for each $i \in [1, 9]$, there are exactly i numbers between two successive occurrences of the number i . Therefore, 1, 2, 1, 8, 2, 4, 6, 2 could be a part of the final solution, while 1, 2, 1, 8, 2, 4, 2, 6 is not (because there is only a single number between the last two occurrences of 2). The reader is invited to try to write a program that solves this problem using her preferred imperative programming language. It is not a difficult exercise but does require some care, because even if the “what” must be done is clear, the “how” of the desired solution is not trivial. For example, in a naive (and inefficient) way, we may generate all possible permutations of a list of 27 numbers containing three 1s, three 2s, three 3s, . . . , three 9s, to see if one of them satisfies the required propriety. Even this solution (probably the conceptually simplest one), however, requires the specification in detail of the aspects of control to generate the permutations of a list.

Reasoning in a declarative fashion, on the other hand, we can proceed as follows. First, we need a list which we will call `LS`. This list will have to contain 27 elements,

something that we can specify using a unary predicate (that is, a relation symbol)¹ `list_of_27`. If we write `list_of_27(Ls)`, we mean, therefore, that `Ls` must be a list of 27 elements. In other words, `list_of_27` defines a unary relation (a subset of all possible finite strings) formed by all the lists of 27 elements. For this, we can define `list_of_27` as follows:

```
list_of_27(Ls):-
    Ls = [_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_].
```

where the part to the left of the `:-` symbol denotes what is being defined, while the part to the right of `:-` indicates the definition. The `=` denotes an equality concept whose definition, for the time being, is left to intuition (it has nothing to do with an assignment, however). Anticipating the PROLOG notation for lists, here we write `[X1, X2, ..., Xn]` to denote the list that contains the n variables `X1`, `X2`, ..., `Xn` (see Sect. 12.4.5 for more details about lists). We use `_` for the *anonymous* variable, that is a variable whose name is of no interest and which is distinct from all other variables present, including all other anonymous variables.

In order to satisfy our specification, the list, `Ls`, in addition to being composed of 27 elements, must contain a sublist² in which the number 1 appears followed by any other number, by another occurrence of the number 1, then another number and finally a last occurrence of the number 1. Such a sublist can be specified as

```
[1, X, 1, Y, 1]
```

where `X` and `Y` are variables. Exploiting again the anonymous variable, we may write:

```
[1, _, 1, _, 1]
```

Assuming that we have available a binary predicate `sublist(X, Y)` whose meaning is that the first argument (`X`) is a sublist of the second (`Y`),³ our requirement therefore can be expressed by writing:

```
sublist([1, _, 1, _, 1], Ls)
```

Moving on to number 2 and reasoning in a similar way, we obtain that the list `Ls` must also contain the sublist:

```
[2, _, _, 2, _, _]
```

and therefore the following must also be true:

```
sublist([2, _, _, 2, _, _], Ls)
```

Repeating this reasoning for all the other numbers up to 9, the `sol` program that we want to produce can be described as follows:

¹ Recall that unary means that it has a single argument while n -ary means that there are n arguments.

² Recall that `Li` is a sublist of `Ls` if `Ls` is obtained by concatenating a (possibly empty) list with `Li` and with another list (itself also possibly empty).

³ The definition of the relation `sublist` is given in Sect. 12.4.5.

1, 9, 1, 2, 1, 8, 2, 4, 6, 2, 7, 9, 4, 5, 8, 6, 3, 4, 7, 5, 3, 9, 6, 8, 3, 5, 7.

Successive calls to the same procedure allow obtaining the other solutions to the problem.

With this procedural interpretation, one has a true programming language that allows us to express, in a compact and relatively simple way, programs that solve even complex problems. For this power, the language pays the penalty of efficiency. In the preceding program, despite its apparent simplicity, the computation performed by the language's abstract machine is complex, given that the interpreter must try the various combinations of possible sublists until it finds the one that satisfies all the given conditions. In these search processes, a *backtracking* mechanism is used. When the computation reaches at a point at which it cannot proceed, the computation that has been performed so far is undone so that a decision point can be reached, if it exists, at which an alternative is chosen that is different from the previous one (if this alternative does not exist, the computation terminates in failure). It is not difficult to see that, in general, this search process can have exponential complexity.

12.2 Syntax

Logic programs are sets of logic formulæ of a particular form. We begin therefore with some basic notions for defining the syntax.

The logic of interest here is first-order logic, also called *predicate calculus*. Symbols are used to express properties of (or, in a more old-fashioned terminology, to “predicate on”) elements of a fixed domain of discourse, $\mathcal{D}$. Higher-order logics (second, third, etc., order) also permit predicates whose arguments are more complicated objects such as sets and functions over $\mathcal{D}$ (second order), sets of these functions (third order), etc., in addition to elements of $\mathcal{D}$.

12.2.1 The Language of First-Order Logic

The language of first-order logic consists of three components:

1. An *alphabet*.
2. *Terms* defined over this alphabet.
3. *Well-formed formulæ* defined over this alphabet.

Alphabet

The alphabet is a set⁵ of symbols, partitioned into two disjoint subsets: the set of *logical symbols* (common to all first-order languages) and the set of *non-logical* (or *extra-logical*) symbols (which are specific to a domain of interest). For example, all

⁵ All the sets we consider here are finite or denumerable.

first-order languages will use a (logical) symbol to denote conjunction. If we are considering orderings on a set, we will probably also have the $<$ symbol as one of the non-logical symbols.

The set of *logical symbols* contains the following elements:

- The logical connectives $\wedge$ (conjunction), $\vee$ (disjunction), $\neg$ (negation), $\rightarrow$ (implication) and $\leftrightarrow$ (double implication).
- The propositional constants *true* and *false*.
- The quantifiers $\exists$ (exists) and $\forall$ (forall).
- Some punctuation symbols such as brackets “(” and “)” and comma “,”.
- An (denumerably) infinite set V of *variables*, written $X, Y, Z, \dots$

The *non-logical* (or *extra-logical*) symbols are defined by a *signature with predicates* (Σ, Π) . This is a pair in which the first element, Σ is the *function signature*, that is a set of function symbols, each considered with its own arity.⁶ The second element of the pair, Π , is the *predicate signature*, a set of predicate symbols together with their arities. Functions of arity 0 are said to be *constants* and are denoted by the letters $a, b, c, \dots$. Function symbols of positive arity are, as usual, denoted by $f, g, h, \dots$, while predicate symbols are denoted by $p, q, r, \dots$. Let us assume that the sets Σ and Π have an empty intersection and are also disjoint from the logical symbols. The difference between function and predicate symbols is that the former must be interpreted as functions, while the latter must be interpreted as relations. This distinction will become clearer when we discuss formulæ.

Terms

The concept of term, which is fundamental to mathematical logic and Computer Science, is used implicitly in many contexts. For example, an arithmetic expression is a term obtained by applying (arithmetic) operators to operands. Other types of construct, too, such as strings, binary trees, lists, and so on, can be conveniently seen as terms which are obtained using appropriate constructors. Semantically, a term denotes an element of the domain of discourse, $\mathcal{D}$.

Syntactically, in the simplest case a term is obtained by applying a function symbol to as many variables and constants as required by its arity. For example, if a and b are constants, X and Y are both variables and f and g have arity 2, then $f(a, b)$ and $g(a, X)$ are terms. Nothing prevents the use of terms as the arguments to a function, provided that the arity is respected. We can, for example, write $g(f(a, b), Y)$ or $g(f(a, f(X, Y)), X)$ and so on.

In the most general case, we can define terms as follows.

Definition 12.1 (*Term*) The terms over a signature Σ (and over the set, V , of variables) are defined inductively⁷ as follows:

⁶ Recall that, as stated above, the arity denotes the number of arguments of a function or relation.

⁷ See the box on Sect. 6.4 for inductive definitions.

- A variable (in V) is a term.
- If f (in Σ) is a function symbol of arity n and $t_1, \dots, t_n$ are terms, then $f(t_1, \dots, t_n)$ is a term.

As a particular case of the second point, a constant is a term. According to the letter of the definition, a term which corresponds to a constant must be written with parentheses: $a()$, $b()$, $\dots$. Let us establish, for ease of reading, that in the case of function symbols of arity 0, parentheses are omitted. Terms without variables are said to be *ground* terms. Terms are usually denoted by the letters $s, t, u, \dots$. Note that predicates do not appear in terms; they appear in formulæ (to express properties of terms).

Formulæ

The *well-formed formulæ* (or *formulæ* for short) of the language allow us to express the properties of terms. Semantically, therefore, a formula states that some elements of our domain, $\mathcal{D}$, enjoy a property. For example, if we have the predicate $>$ (interpreted as usual as a partial order relation on $\mathcal{D}$), writing $>(3, 2)$ we express that the term “3” corresponds to a value (an element of $\mathcal{D}$) which is greater than the value associated with the term “2”. More complex formulæ can be constructed using logical symbols. For example, the formula $>(X, Y) \wedge >(Y, Z) \rightarrow >(X, Z)$ expresses the transitivity of $>$ —if $>(X, Y)$ is true and $>(Y, Z)$ is also true then, it is the case that $>(X, Z)$.

Wanting to define formulæ precisely, we have first atomic formulæ (or atoms), constructed by the application of a predicate to the number of terms required by its arity. For example, if p has arity 2, using the two terms introduced above, we can write $p(f(a, b), f(a, X))$. Using logical connectives and quantification, we can construct complex formulæ from atomic ones. As usual, we have an inductive definition (or, equivalently, a free grammar—see Exercise 1).

Definition 12.2 (*Formula*) The (well-formed) formulæ over the signature with terms (Σ, Π) are defined inductively as follows:

1. If $t_1, \dots, t_n$ are terms over the signature Σ and $p \in \Pi$ is a predicate symbol of arity n , then $p(t_1, \dots, t_n)$ is a formula.
2. *true* and *false* are formulæ.
3. If F and G are formulæ, then $\neg F$, $(F \wedge G)$, $(F \vee G)$, $(F \rightarrow G)$ and $(F \leftrightarrow G)$ are formulæ.
4. If F is a formula and X is a variable, then $\forall X.F$ and $\exists X.F$ are formulæ.

12.2.2 Logic Programs

A formula constructed according to the last definition can have a highly complex structure. This complexity often determines also the effort required to find a proof. In

automatic theorem proving, and in logic programming, particular classes of formulæ have been identified, which lend to more efficient manipulation. One of this classes is the one of the *clauses*, which can be manipulated with a special inference rule called *resolution*. We are interested in a restriction of the notion of clause, called *definite clause*, as well as in a restricted forms of resolution—*SLD resolution*, see Sect. 12.4.3. Using these two notions, the procedure for seeking a proof is not only particularly simple but also allows the explicit calculation of the values of the variables necessary for the proof. These values can be considered as the result of the computation, giving way to an interesting model of computation based on logical deduction. We will see this model in more detail below, for now we will concentrate on syntactic aspects.

Definition 12.3 (*Logic Program*) Let $H, A_1, \dots, A_n$ be atomic formulæ. A definite clause (for us simply a “clause”) is a formula of the form:

$$H : -A_1, \dots, A_n.$$

If $n = 0$, the clause is said to be a *unit*, or a fact, and the symbol $: -$ is omitted (but not the final full stop). A logic program is a set of clauses, while a pure PROLOG program is an ordered sequence of clauses. A query (or goal) is a sequence of atoms $A_1, \dots, A_n$.

Let us clarify some points about this definition. First, the symbol $: -$ which we did not include in our alphabet, is just (reversed) implication ($\leftarrow$).⁸

The commas in a clause or in a query should be interpreted as logical conjunction. The notation “ $H : -A_1, \dots, A_n$.” is therefore an abbreviation for “ $H \leftarrow A_1 \wedge \dots \wedge A_n$.” Note that the full stop is part of the notation and is important because it tells a potential interpreter or compiler that the clause has terminated.

The part on the left of $: -$ is the *head* of the clause; that on the right is the *body*. A fact is therefore a clause with an empty body. A program is a set of clauses in the case of the theoretical formalism. In the case of PROLOG, on the other hand, a program is considered as a sequence because, as we will see, the order of clauses is relevant. Here, we used the simplified terminology found in many recent texts (for example, [1]). For more precise terminology, see the next box. The set of clauses containing the predicate symbol p in their head is said to be the *definition* of p . Variables occurring in the body of a clause and not in the head are said to be *local* variables.

⁸ $: -$ instead of $\leftarrow$ is used for practical reasons. When logic languages were being introduced, it was much easier to type $: -$ rather than a left arrow, on most keyboards.

Clause

The reader familiar with first-order logic will have recognised the notion of clause given in Definition 12.3 as being a particular case of the one used in logic. A clause, in the general sense, is indeed a formula of the form:

$$\forall X_1, \dots, X_m (L_1 \vee L_2 \vee \dots \vee L_n)$$

where $L_1 \dots L_n$ are *literals* (atoms or negated atoms) and $X_1, \dots, X_m$ are all the variables that occur in $L_1 \dots L_n$. For clarity, we may separate the negated atoms from the others, therefore writing a clause as:

$$\forall X_1, \dots, X_m (A_1 \vee A_2 \vee \dots \vee A_m, \neg B_1 \vee \neg B_2 \vee \dots \vee \neg B_k).$$

Using well-known logical equivalences, we can express this formula in the following equivalent form;

$$A_1, \dots, A_m \leftarrow B_1, \dots, B_k. \quad (12.1)$$

A *program clause*, also called a *definite clause*, is a clause that has only one un-negated atom. In the form shown in (12.1), a definite clause always has $m = 1$, which is the notion introduced directly in Definition 12.3. A fact is therefore a definite clause containing no negated atoms. Finally, a *negative clause*, also called a *query* or *goal*, is a clause of the form (12.1) in which $m = 0$.

12.3 Theory of Unification

The fundamental computational mechanism in logic programming is the solution of equations between terms using the unification procedure. In this procedure, substitutions are computed so that variables and terms can be bound (or instantiated). The composition of the different substitutions obtained in the course of a computation provides the result of the calculation. Before seeing the computational model for logic programming in detail, we must analyse unification in a little detail, a task we undertake in this section.

12.3.1 The Logic Variable

Let us clarify, first, that the concept of variable we are considering is different from that seen in Sect. 6.2.1. Here, we consider the so-called *logic variable*, an unknown which can assume values from a predetermined set. In our case, this set is that of definite terms over the given alphabet. This fact, together with the use that logic programming makes of logic variables, gives rise to three important differences between logic variables and modifiable variables of imperative languages.

1. The logic variable can be bound only once, in the sense that if a variable is bound to a term, this binding cannot be destroyed (but the term might be modified, as will be explained below). For example, if we bind the variable X to the constant a in a logic program, the binding cannot later be replaced by another which binds X to the constant b . Clearly, this is possible in imperative languages using assignment. The fact that the binding of a variable cannot be eliminated does

not mean, however, that it is impossible to modify the value of the variable. This apparent contradiction is dealt with in the next point.

2. The value of a logic variable can be partially defined (or can be undefined), to be specified later. This is because a term that is bound to a variable can contain, in its turn, other logic variables. For example, if the variable X is bound to the term $f(Y, Z)$, successive bindings of the variables Y and Z will also modify the value of the variable X : if Y is bound to a and Z is bound to $g(W)$, the value of X will become the term $f(a, g(W))$. The process could continue by modifying the value of W . This mechanism for specifying the value of a variable by successive approximations, so to speak, is typical of logic languages and is somewhat different from the corresponding one encountered in imperative languages, where a value assigned to a value cannot be partially defined.
3. A third important difference concerns the bidirectional nature of bindings for logic variables. If X is bound to the term $f(Y)$ and later we are able to bind X to the term $f(a)$, the effect so produced is that of binding the variable Y to the constant a . This does not contradict the first point, given that the binding of X to the term $f(Y)$ is not destroyed, but the value of $f(Y)$ is specified through the binding of Y . Therefore, we can not only modify the value of a variable by modifying the term to which it is bound, but we can also modify this term by providing another binding for that variable. Clearly, this second binding must be consistent with the first, that is if X is bound to the term $f(Y)$, we cannot try to bind X to a term of the form $g(Z)$.

The last point is fundamental, since it allows us to use a single logic program in quite different ways, as will be seen below. Essentially, we are talking about a property that derives from the unification mechanism, which we will discuss shortly. First, however, we must introduce the concept of substitution.

12.3.2 Substitution

The connection between variables and terms is made in terms of the concept of substitution, which, as its name tells us, allows the “substitution” of a variable by a term. A substitution, usually denoted by the greek letters $\vartheta, \sigma, \rho, \dots$, can be defined as follows.

Definition 12.4 (*Substitution*) A substitution is a function from variables to terms such that the number of variables which are not mapped to themselves is finite. We denote a substitution ϑ by the notation:

$$\vartheta = \{X_1/t_1, \dots, X_n/t_n\}$$

where $X_1, \dots, X_n$ are different variables, $t_1, \dots, t_n$ are terms and where we assume that t_i is different from X_i , for $i = 1, \dots, n$.

In the preceding definition, a pair X_i/t_i is said to be a *binding*.⁹ In the case in which all the $t_1, \dots, t_n$ are ground terms, then ϑ is said to be a *ground substitution*. We write ϵ for the empty substitution. For ϑ represented as in Definition 12.4, we define the domain, codomain and variables of a substitution as follows:

$$\text{Domain}(\vartheta) = \{X_1, \dots, X_n\},$$

$$\text{Codomain}(\vartheta) = \{Y \mid Y \text{ a variable in } t_i, \text{ for some } t_i, 1 \leq i \leq n\}.$$

A substitution can be applied to a term, or, more generally, to any syntactic expression, to modify the value of the variables present in the domain of the substitution. More precisely, if we consider an expression, E (which could be a term, a literal, a conjunction of atoms, etc.), the result of the application of $\vartheta = \{X_1/t_1, \dots, X_n/t_n\}$ to E , denoted by $E\vartheta$, is obtained by simultaneously replacing every occurrence of X_i in E by the corresponding t_i , for all $1 \leq i \leq n$. For example, if we apply the substitution $\vartheta = \{X/a, Y/f(W)\}$ to the term $g(X, W, Y)$, we obtain the term $g(X, W, Y)\vartheta$, that is $g(a, W, f(W))$. Note that the application is simultaneous. For example, if we apply the substitution $\sigma = \{Y/f(X), X/a\}$ to the term $g(X, Y)$, we obtain $g(a, f(X))$ (and not $g(a, f(a))$).

The *composition*, $\vartheta\sigma$, of two substitutions $\vartheta = \{X_1/t_1, \dots, X_n/t_n\}$ and $\sigma = \{Y_1/s_1, \dots, Y_m/s_m\}$ is defined as the substitution obtained by removing from the set

$$\{X_1/t_1\sigma, \dots, X_n/t_n\sigma, Y_1/s_1, \dots, Y_m/s_m\}$$

the pairs $X_i/t_i\sigma$ such that X_i is equal to $t_i\sigma$ and the pairs Y_i/s_i such that $Y_i \in \{X_1, \dots, X_n\}$. Composition is associative and it is not difficult to see that, for any expression, E , it is the case that $E(\vartheta\sigma) = (E\vartheta)\sigma$. The effect of the application of a composition is the same as it is obtained by successively applying the two substitutions that we want to compose.

For example, composing

$$\vartheta_1 = \{X/f(Y), W/a, Z/X\} \quad \text{and} \quad \vartheta_2 = \{Y/b, W/b, X/Z\}$$

we obtain the substitution

$$\vartheta = \vartheta_1\vartheta_2 = \{X/f(b), W/a, Y/b\}.$$

If we apply the latter to the term $g(X, Y, W)$, we obtain the term $g(f(b), b, a)$. The same term is obtained first by applying ϑ_1 to $g(X, Y, W)$, then applying ϑ_2 to the result. Note that, in the result ϑ of the composition, the Y in ϑ_1 is instantiated to b because of the binding occurring in ϑ_2 . The X occurring in Z/X is instantiated to Z using the binding X/Z in ϑ_2 , after which the binding Z/Z is eliminated from the resulting substitution (because it is the identity). The bindings W/b and X/Z

⁹ We are using here an “opposite” notation of the one we used in the context of the λ -calculus, Sect. 11.6. There, according to the prevalent tradition in functional languages, we wrote N/X for the substitution of the term N in place of the variable X . Here such a binding is written X/N (i.e., backwards). In this chapter we will use the latter notation, which is the one most commonly used in logic programming.

present in ϑ_2 finally disappear from ϑ because both W and Z appear in the domain (or, on the left of a binding) in ϑ_1 .

A particular type of substitution is formed from those which simply rename their variables. For example, the substitution $\{X/W, W/X\}$ does nothing more than change the names of the variables X and W . Substitutions like this are called *renamings* and can be defined as follows.

Definition 12.5 (*Renaming*) A substitution ρ is a renaming if its inverse substitution ρ^{-1} exists and is such that $\rho\rho^{-1} = \rho^{-1}\rho = \epsilon$.

Note that the substitution $\{X/Y, W/Y\}$ is not a renaming. Indeed, it not only changes the names of the two variables, but also makes the two variables equal, while they were previously distinct.

Finally, it will be useful to define a preorder, $\leq$, over substitutions, where $\vartheta \leq \sigma$ is read as: ϑ is more general than σ . Let us therefore define $\vartheta \leq \sigma$ if (and only if) there exists a substitution γ such that $\vartheta\gamma = \sigma$. Analogously, given two expressions, t and t' , we define $t \leq t'$ (t is more general than t') if and only if there exists a ϑ such that $t\vartheta = t'$. The relation $\leq$ is a preorder and the equivalence induced by it¹⁰ is called the *variance*; t and t' are therefore variants if t is an instance of t' and, conversely, t' is an instance of t . It is not difficult to see that this definition is equivalent to saying that t and t' are variants if there exists a renaming ρ such that t is syntactically identical to $t'\rho$. These definitions can be extended to any expression in an obvious fashion. Finally, if ϑ is a substitution that has as domain the set of variables V , and W is a subset of V , the *restriction* of ϑ to the variables in W is the substitution obtained by considering only the bindings for variables in W , that is the substitution defined as follows:

$$\{Y/t \mid Y \in W \text{ and } Y/t \in \vartheta\}.$$

A comparison with the imperative paradigm can help in better understanding the concepts under consideration. As we saw in Sect. 2.5, in the imperative paradigm, the semantics can be expressed by referring to a concept of state that associates every variable with a value.¹¹ An expression containing variables is evaluated with respect to a state to obtain a value that is completely defined.

In the logic paradigm, the association of values with variables is implemented through substitutions. The application of a substitution to a term (or to a more complex expression) can be seen as the evaluation of the terms, an evaluation that returns another term, and therefore, in general, a partially defined value.

¹⁰ Given a preorder, $\leq$, the equivalence relation *induced* by $\leq$ is defined as $t = t'$ if and only if $t \leq t'$ and $t' \leq t$.

¹¹ Wishing to be precise, as we have seen in the box on Sect. 6.3, in real languages, this association is implemented using two functions, environment and memory. This however does not alter the import of what we are saying.

12.3.3 Most General Unifier

The basic computation mechanism for the logic paradigm is the evaluation of equations of the form $s = t$,¹² where s and t are terms and “=” is a predicate symbol interpreted as syntactic equality over the set of all ground terms; this set is called the *Herbrand Universe*.¹³ We will attempt better to clarify this equality.

If we write $X = a$ in a logic program, we mean that the variable X must be bound to the constant a . The substitution $\{X/a\}$ therefore is a solution to this equation since, by applying the substitution to the equation, we obtain $a = a$ which is a syntactical identity. You should not be deceived by the syntactic analogy with assignment in imperative languages, for it deals with a completely different concept. Indeed, unlike in an imperative language, here we can also write $a = X$ instead of $X = a$ and the meaning does not change (the equality that we are considering is symmetric, as are all equality relations). Also the analogy with the equality of arithmetic expressions can be, in some ways, misleading, as illustrated by the following example.

Let us assume that we have a binary function symbol $+$ which, intuitively, expresses the sum of two natural numbers, and consider the equation $3 = 2 + 1$, where, for simplicity, we use infix notation for $+$ and we represent in the usual fashion the natural numbers. Given that the equation $3 = 2 + 1$ does not contain variables, it can be either true (or, solved) or false (that is, insoluble). Contrary to what arithmetic intuition would suggest to us, in a (pure) logic program this equation cannot be solved. This is because, as we have said, the symbol $=$ is interpreted as syntactic equality over the set of ground terms (the Herbrand universe). It is clear that, from the syntactic viewpoint, the constant 3 is different from the term $2 + 1$ and since they are treated as ground terms (that is completely instantiated terms) there is no way that they can be made syntactically equal. Analogously, the equation $f(X) = g(Y)$ has no solutions (it is not solvable) because however the variables X and Y are instantiated, we cannot make the two different function symbols, f and g , equal. Note that the equation, $f(X) = f(g(X))$, also has no solutions, because the variable X in the left-hand term must be instantiated with $g(X)$ and therefore the possible solution must contain the substitution $\{X/g(X)\}$. The application of this substitution to the term on the right would instantiate X , producing the term $f(g(g(X)))$, so the X in the right-hand term would have to be instantiated to $g(g(X))$ rather than to $g(X)$ and so on, without ever reaching a solution.¹⁴ In general, therefore, the equation $X = t$ cannot be solved if t contains the variable X (and t is different from X).

¹² We draw attention to the notation that, as usual, overloads the “=” symbol. By writing $\vartheta = \{X_1/t_1, \dots, X_n/t_n\}$, we mean that ϑ is the substitution $\{X_1/t_1, \dots, X_n/t_n\}$, while writing $s = t$, we mean an equation.

¹³ The symbol “=” is usually written in infix notation for increased readability. Different logic languages can have different syntactic readings for it and use different equality symbols, each with a different meaning. Here, we refer to “pure” logic programming.

¹⁴ By admitting infinite terms, we can find a solution, however.

On the other hand, the equation $f(X) = f(g(Y))$ is solvable. One solution is the substitution $\vartheta = \{X/g(Y)\}$ because if this is applied to the two terms in the equation, it makes them syntactically equal. Indeed, $f(X)\vartheta$ is identical to $f(g(Y))$ which is identical to $f(g(Y))\vartheta$. In more formal terms, we can say that the substitution ϑ *unifies* the two terms of the equation— ϑ is a *unifier*. We have said “a solution” because there are many substitutions (an infinite number of them) that are unifiers of X and $g(Y)$. It is sufficient to instantiate Y in the definition of ϑ . So, for example, the substitution $\{X/g(a), Y/a\}$ is also an unifier, as are $\{X/g(f(Z)), Y/f(Z)\}$, $\{X/g(f(a)), Y/f(a)\}$, and so on. All these substitutions are, however, *less general* than ϑ according to the preorder that we defined above. Each of them, that is, can be obtained by composing ϑ with some other, appropriate, substitution. For example, $\{X/g(a), Y/a\}$ is equal to $\vartheta\{Y/a\}$ (remember that we denote the composition of substitutions with juxtaposition). In this sense, we say that ϑ is the most general unifier (or m.g.u.) of X and $g(Y)$.

Before moving to general definitions, note one last important detail: the process of solving an equation (the unification process), can create bidirectional bindings, that is the direction in which the associations must be realised is not specified. For example, a solution of the equation $f(X, a) = f(b, Y)$ is given by the substitution $\{X/b, Y/a\}$, where a variable on the left and one on the right of the $=$ symbol are bound. Instead, in the equation, $f(X, a) = f(Y, a)$, to find a solution, we can choose whether to bind the variable on the left (using $\{X/Y\}$) or the one on the right (using $\{Y/X\}$) or even both (using $\{X/Z, Y/Z\}$).

This aspect, to which we will return, is important because it allows the implementation of bidirectional parameter-passing mechanisms. It allows also a unique characteristic of the logic paradigm—to use the same program in different ways, turning input arguments into outputs and vice versa, without modifying the program.

We may now give the formal definition.

Definition 12.6 (*M.g.u.*) Given a set of equations $E = \{s_1 = t_1, \dots, s_n = t_n\}$, where $s_1, \dots, s_n$ and $t_1, \dots, t_n$ are terms, the substitution, ϑ , is a unifier for E if the sequence $(s_1, \dots, s_n)\vartheta$ and $(t_1, \dots, t_n)\vartheta$ are syntactically identical. A unifier of E is said to be the *most general unifier* (m.g.u.) if it is more general than any other unifier of E ; that is, for every other unifier, σ , of E , σ is equal to $\vartheta\tau$ for some substitution, τ .

The preceding concept of unifier can be extended to other syntactic objects in an obvious fashion. In particular, we say that ϑ is a unifier of two atoms $p(s_1, \dots, s_n)$ and $p(t_1, \dots, t_n)$ if ϑ is a unifier of $\{s_1 = t_1, \dots, s_n = t_n\}$.

12.3.4 A Unification Algorithm

An important result, due to Robinson in 1965, shows that the problem of determining whether a set of equations of terms can be unified is decidable. The proof is constructive, in the sense that it provides a *unification algorithm* which, for every set of equations, produces their m.g.u. if the set is unifiable and returns a failure in the opposite case.¹⁵

It can also be proved that a m.g.u. is unique up to renaming. The unification algorithm which we will now see is not Robinson's original but is Martelli and Montanari's from 1982 and it makes use of ideas present in Herbrand's thesis of 1930.

Martelli and Montanari's Unification Algorithm

Given a set of equations

$$E = \{s_1 = t_1, \dots, s_n = t_n\},$$

the algorithm produces either failure or a set of equations in the following, so called, solved form:

$$\{X_1 = r_1, \dots, X_m = r_m\},$$

where $X_1, \dots, X_m$ are distinct variables not appearing in the terms $r_1, \dots, r_m$. The set of equations is equivalent to the starting set, E , and from it we can obtain a m.g.u. for E simply interpreting every equality as a binding. Therefore, the m.g.u. that we seek is the substitution:

$$\{X_1/r_1, \dots, X_m/r_m\}.$$

The algorithm is non-deterministic in the sense that when there are more possible actions, one is chosen in an arbitrary fashion, with no priority between the various actions.¹⁶ The algorithm is given by the following steps.

1. Nondeterministically select one equation from the set E .
2. According to the type of equation chosen, execute, if possible, one of the specific operations as follows (where on the left of the ":" we indicate the type of equation and, on the right, the associated action):
 - a. $f(l_1, \dots, l_k) = f(m_1, \dots, m_k)$: eliminate this equation from the set E and add to E the equations $l_1 = m_1, \dots, l_k = m_k$.
 - b. $f(l_1, \dots, l_k) = g(m_1, \dots, m_k)$: if f is different from g , terminate with failure.

¹⁵ Robinson's original algorithm considers the unification of just two terms but this, obviously, is not reductive given that the unification of $\{s_1 = t_1, \dots, s_n = t_n\}$ can be seen as the unification of $f(s_1, \dots, s_n)$ and $f(t_1, \dots, t_n)$.

¹⁶ We will return to the non-determinism briefly when we discuss the operational semantics of logic programs.

- c. $X = X$: eliminate this equation from the set E .
 - d. $X = t$: if t does not contain the variable X and this variable appears in another equation in the set E , apply the substitution $\{X/t\}$ to all the other equations in the set E .
 - e. $X = t$: if t contains the variable X , terminate with failure.
 - f. $t = X$: if t is not a variable, eliminate this equation from the set E and add to E the equation $X = t$.
3. If none of the preceding operations is possible, terminate with success (E contains the solved form). If, on the other hand, an operation different from termination with failure has been executed, go to (1).

This is a simple algorithm, but it is worth discussing in detail its steps.

In the first case the two terms agree on the function symbol. To unify the two terms, it is necessary to unify the arguments, so we replace the original equation with the equations obtained by equating the arguments in each position. Note that this case also includes equivalence between constants of the form $a = a$ (where a is a function symbol of arity 0) which are eliminated without adding anything.

The second case produces a failure given that, when f and g are different, the two terms cannot be unified.

The equation $X = X$ is eliminated using the identity substitution which therefore produces no change in the other equations.

The fourth case is the most interesting. An equation $X = t$ is already in solved form (because, by assumption, t does not contain X). In other words, the substitution $\{X/t\}$ is the m.g.u. of this very equation. We need to combine the effect of such an m.g.u. with those produced by other equations, so we apply the substitution $\{X/t\}$ to all the other equations in E .

On the other hand, in the case in which t contains the variable X , as we have already seen, the equation has no solution and therefore the algorithm terminates with failure. It is important to note that this check, called the *occurs check*, is removed from many implementations of PROLOG for reasons of efficiency. Therefore, many PROLOG implementations use an incorrect unification algorithm!

The last case, finally, serves only to obtain a result form in which the variables appear on the left and the terms on the right of the $=$ symbol.

It is easy to convince oneself that the algorithm terminates, given that the depth of the input terms is finite. It is, moreover, possible to prove that the algorithm produces an m.g.u. which is obtained by the interpretation as substitutions of the final result form of the equations.

By a careful consideration of the algorithm, it can be seen that the computation of the most general unifier occurs in an incremental fashion by solving ever simpler equations until a result form is encountered. It is also possible to express this process in terms of substitution compositions as happens in the operational model of logic languages. We will exemplify this point with an example. To simplify, we will always choose the leftmost equation (the final result, anyway, does not depend upon such an assumption; any other selection rule would lead to the same result up to renaming).

Let us consider the set of equations:

$$E = \{f(X, b) = f(g(Y), W), h(X, Y) = h(Z, W)\}.$$

Choosing the first equation on the left, using the operation described in (a), the set E is transformed into:

$$E_1 = \{X = g(Y), b = W, h(X, Y) = h(Z, W)\}.$$

Using operation (d), we therefore obtain:

$$E_2 = \{X = g(Y), b = W, h(g(Y), Y) = h(Z, W)\}.$$

Using (f) and then (d) again on the second equation, we finally obtain:

$$E_3 = \{X = g(Y), W = b, h(g(Y), Y) = h(Z, b)\}.$$

This already contains the result of the first equation in the set E . In fact, the substitution:

$$\vartheta_1 = \{X/g(Y), W/b\}$$

is an m.g.u. for $f(X, b) = f(g(Y), W)$.

Continuing with the second equation from the original set, suitably instantiated by the substitutions already computed, using operation (a), we obtain:

$$E_4 = \{X = g(Y), W = b, g(Y) = Z, Y = b\};$$

then, by (f), we obtain:

$$E_5 = \{X = g(Y), W = b, Z = g(Y), Y = b\}.$$

Finally, using (d) applied to the last equation, we have:

$$E_4 = \{X = g(b), W = b, Z = g(b), Y = b\},$$

which is the result form for the set E and therefore also provides the m.g.u. of the initial set in the form of the substitution

$$\vartheta = \{X/g(b), W/b, Z/g(b), Y/b\}.$$

Two important observations can be made. First, note how the value of some variables can be partially specified first, and then later refined. For example, ϑ_1 (m.g.u. of the first equation in E) tells us that X has $g(Y)$ as its value and only by solving the second equation do we see that Y has b as its value. Therefore it can be seen that $g(b)$ is the value of X (as, indeed, it results in the final m.g.u.).

Moreover, we can see that if we consider $\{h(g(X), Y) = h(Z, W)\}\vartheta_1$ (the second equation in E instantiated using the m.g.u. of the first), we obtain the equation

$$h(g(Y), Y) = h(Z, b)$$

for which the substitution

$$\vartheta_2 = \{Z = g(b), Y = b\}$$

is a m.g.u. Using the definition of composition of substitutions, it is easy to check that $\vartheta = \vartheta_1\vartheta_2$. The m.g.u. of the set E can therefore be obtained by composing the first equation's m.g.u. with that of the second (to which the first m.g.u. has already been applied). This, as we have already said, is what, indeed, normally happens in implementations of logic languages, where, instead of accumulating all the equations and then solve them, an m.g.u. is computed on each step of the computation and is composed with the ones that were previously obtained.

12.4 The Computational Model

The logic paradigm, implementing the idea of “computation as deduction”, uses a computational model that is substantially different from all the others that we have seen so far. Wishing to synthesise, we can identify the following main differences from the other paradigms:

1. The only possible values, at least in the pure model, are terms over a given signature.
2. Programs can have a declarative reading which is entirely logical, or a procedural reading of an operational kind.
3. Computation works by instantiating the variables appearing in terms (and therefore in goals) to other terms using the unification mechanism.
4. Control, which is entirely handled by the abstract machine (except for some possible annotations in PROLOG) is based on the process of automatic backtracking.

Below, we analyse these four points. We will explicitly discuss the differences between logic programming and PROLOG.

12.4.1 The Herbrand Universe

In logic programming, terms are a fundamental element. The set of all possible terms over a given signature is called the *Herbrand Universe* and is the domain over which computation in logic programs is performed. It has some characteristics that must be understood.

- The alphabet over which programs are defined is not fixed but can vary as far as non-logical symbols are concerned.
- As a (partial) consequence of the previous point, unlike what happens in imperative languages, no predefined meaning is assigned to the (non-logical) symbols of the alphabet. For example, a program can use the $+$ symbol to denote addition, while another program can use the same symbol to denote string concatenation. The exceptions are the (binary) equality predicate and some other predefined (“built-in”) symbols in PROLOG.¹⁷
- As a final consequence, no type system is present in logic languages (at least in the classic formalism). The only type that is present is that of terms with which we can represent arithmetic, list, expressions, etc.

¹⁷ As well as predefined predicates in constraint languages, as we will see in the next chapter.

From the theoretical stance, the fact that there are no types and that computation occurs in the Herbrand Universe is not limiting, rather it permits the highly elegant and, all considered, simple expression of the formal semantics of logic programs. For example, with only two function symbols, 0 (constant zero) and s (successor, of arity 1), we can express the natural numbers using the terms 0, $s(0)$, $s(s(0))$, $s(s(s(0)))$, etc. With a little effort, we can express the normal arithmetic operations in terms of this two-symbol representation.

From the practical view point, on the other hand, the lack of types is a serious problem. In fact, in PROLOG, as in other, more recent, logic languages, some primitive types have been introduced (for example, integers and associated arithmetic operations). The languages of this paradigm are therefore always somewhat lacking as far as types are concerned.

12.4.2 Declarative and Procedural Interpretation

As we have just hinted, a clause, and therefore a logic program, can have two different interpretations: one *declarative* and one *procedural*.

From the *declarative* viewpoint, a clause $H : -A_1, \dots, A_n$ is a formula which expresses that if A_1 and A_2 and $\dots$ and A_n are true, then H is also true. A query (or goal) is also a formula for which we want to prove, provided that it is appropriately instantiated, that it is a logical consequence of the program, and is, therefore, true in all interpretations in which the program is true.¹⁸ This interpretation can be developed using the methods of logic (in particular, some elementary concepts of model theory) in such a way as to give a meaning to a program in purely declarative terms without referring at all to a computational process. For this interpretation, while interesting, we refer the reader to the specialist literature cited at the end of the chapter.

The *procedural* interpretation, on the other hand, allows us to read a clause such as:

$$H : -A_1, \dots, A_n$$

as follows. To prove H , it is necessary first to prove $A_1, \dots, A_n$, or rather to compute H , it is necessary first to compute $A_1, \dots, A_n$. From this, we can view a predicate as the name of a procedure, whose defining clauses constitute its body. In this interpretation, we can read an atom in the body of a clause, or in a goal, as a procedure call. A logic program is therefore a set of declarations and a goal is no more than the equivalent of “main” in an imperative program, given that it contains all the calls to the procedures that we want to evaluate. The comma in the body of clauses and in goals, in PROLOG (but not in other pure logic programming languages), can be read as the analogue of “;” in imperative languages.

¹⁸ Here we will content ourselves with an intuitive idea of this concept. The interested reader can consult any text on logic for more information.

Precise correspondence theorems allow us to reconcile the declarative and procedural views, proving that the two approaches are equivalent.

From a formal viewpoint, the procedural interpretation is supported by so-called SLD resolution, a logical inference rule which we will discuss in the box on Sect. 12.4.3. It is also possible to describe the procedural interpretation in a more informal manner, using only the analogy with procedure calls and parameter passing which we have just outlined. This is the approach we will use below.

12.4.3 Procedure Calls

Let us consider for now a simplified definition of clause in which we assume that, in the head, all the arguments of the predicate are distinct variables. An arbitrary clause of this type therefore has the form:

$$p(X_1, \dots, X_n) : -A_1, \dots, A_m$$

and, as we have anticipated, it can be seen as the declaration of the procedure p with n formal parameters, $X_1, \dots, X_n$. An atom $q(t_1, \dots, t_n)$ can be seen as a call to the procedure q with n actual parameters, $t_1, \dots, t_n$. In the definition of p , therefore, the body is formed from the calls to m procedures which constitute the atoms $A_1, \dots, A_m$.

In accordance with this view, and analogously to what happens in imperative languages, the evaluation of the call $p(t_1, \dots, t_n)$ causes the evaluation of the body of the procedure after parameter passing has been performed. Parameter passing uses a technique similar to call by name, replacing the formal parameter X_i with the corresponding actual parameter t_i . Moreover, given that the variables appearing in the body of the procedure are to be considered as logical variables, they can be considered to be distinct from all other variables. In block-structured languages, this happens implicitly given that the body of the procedure is considered as a block with its own local environment. Here, on the other hand, the concept of block is absent, so, in order to avoid conflicts between variable names, we assume that, before using a clause, all the variables appearing in it are systematically renamed (so that they do not conflict with any others).

Using more precise terms, we can say that the evaluation of the call $p(t_1, \dots, t_n)$, with the definition of p seen above, causes the evaluation of the m calls:

$$(A_1, \dots, A_m)\vartheta$$

present in the body of p , appropriately instantiated by the substitution

$$\vartheta = \{X_1/t_1, \dots, X_n/t_n\}$$

which performs parameter passing. In the case in which the body of the clause is empty (or that $m = 0$), the procedure call terminates immediately. Otherwise, the

computation proceeds with the evaluation of the new calls. Using logic programming terminology, this can be expressed by saying that the evaluation of the goal $p(t_1, \dots, t_n)$ produces the new goal:

$$(A_1, \dots, A_m)\{X_1/t_1, \dots, X_n/t_n\}$$

which, in its turn, will have to be evaluated. When all the calls generated by this process have been evaluated (provided there has been no failure), the computation terminates with success and the final result is composed of the substitution that associates the values computed in the course of the computation with the variables present in the initial call ($X_1, \dots, X_n$, in our case). This substitution is said to be the *computed answer* for the initial goal in the given program. We will see a more precise definition of this concept below. For now, we will see a simple example. Let us consider the procedure `list_of_2` defined below and identical to the procedure `list_of_27` of Sect. 12.1.1, except for the lesser number of anonymous variables:

```
list_of_2(Ls):- Ls = [_,_].
```

The evaluation of the call `list_of_2(LXs)` causes the evaluation of the body of the clause, instantiated by the substitution $\{Ls/LXs\}$, that is:

```
LXs = [_,_]
```

This is a particular call because `=` is a predefined predicate which, as we saw in the previous section, is interpreted as syntactic equality over the Herbrand Universe and, operationally, corresponds to the unification operation. The previous call therefore reduces to the attempt (performed by the language interpreter) to solve the equation using unification. In our case, clearly, this attempt succeeds and produces the m.g.u. $\{LXs/[_,_]\}$ which is the result of the computation. The previous substitution is therefore the computed answer for the goal `list_of_2` in the logic program defined by the single line (1) above.

Evaluation of a Non-atomic Goal

The view presented in the previous subsection must be generalised and made more precise to clarify the logic-programming computational model. Below, in order to conform with current terminology, we will talk of atomic goals and goals rather than procedure calls and sequences of calls. The analogy with the conventional paradigm remains valid, though.

In the case in which a non-atomic goal must be evaluated, the computational mechanism is analogous to the one seen above, except that now we must choose one of the possible calls using some *selection rule*. While in the case of pure logic programming no such rule is specified, PROLOG adopts the rule that the leftmost atom is always chosen. It is however possible to prove that whatever rule is adopted, the results that are computed are always the same (see also Exercises 13 and 14 at the end of the chapter).

Assuming, for simplicity, that we adopt the PROLOG rule, we can describe the progress of the evaluation process. Let

$$B_1, \dots, B_k$$

with $k \geq 1$, be the goal to be evaluated. We distinguish the following cases according to the form of the selected atom, B_1 :

1. If B_1 is an equation of the form $s = t$, try to compute a m.g.u. (using the unification algorithm). There are two possibilities:

- a. If the m.g.u. exists and is the substitution σ , then the result of the evaluation is the goal:

$$(B_2, \dots, B_k)\sigma$$

obtained from the previous one by eliminating the chosen atom and applying the m.g.u. thus computed. If $k = 1$ (and therefore $(B_2, \dots, B_k)\sigma$ is empty), the computation terminates in success.

- b. If the m.g.u. does not exist (or the equation has no solutions), then we have failure.

2. If, on the other hand, B_1 has the form $p(t_1, \dots, t_n)$, we have the following two cases:

- a. If, in the program, there exists a clause of the form:

$$p(X_1, \dots, X_n) : -A_1, \dots, A_m$$

(which we consider renamed to avoid variable capture), then the result of evaluation is a new goal:

$$(A_1, \dots, A_m)\vartheta, B_2, \dots, B_k$$

where $\vartheta = \{X_1/t_1, \dots, X_n/t_n\}$. If $k = 1$ (then we have an atomic goal) and $m = 0$ (the body of the clause is empty), then the computation terminates with success.

- b. If, in the program, there exists no clause defining the predicate p , we have failure.

To be able exactly to define the results of the computation (the computed answers), we need to clarify some aspects of control, which we will do in Sect. 12.4.4.

SLD Resolution

Definite clauses allow a natural procedural reading based on *resolution*, an inference rule which is complete for sets of clauses and which was introduced by Robinson and used in automated deduction. In logic programming, we have *SLD resolution*, that is linear resolution guided by a clause selection rule (SLD is an acronym for Selection rule-driven Linear resolution for Definite clauses).

This rule can be described as follows. Let G be the goal $B_1, \dots, B_k$ and let C be the (definite) clause $H : -A_1, \dots, A_n$. We say that G' is *derived* from G and C using ϑ or, equivalently, G' is a *resolvent* of G and C if (and only if) the following conditions are met:

1. B_m , with $1 \leq m \leq k$, is a *selected* atom from those in G .
2. ϑ is the m.g.u. of B_m and H .
3. G' is the goal $(B_1, \dots, B_{m-1}, A_1, \dots, A_n, B_{m+1}, \dots, B_k)\vartheta$.

Note that, unlike Sect. 12.4.3, here it is necessary also to apply ϑ to the other atoms occurring in the goal G because the heads of the clauses contain arbitrary terms and therefore ϑ could instantiate variables that are also in the goal.

Given a goal, G , and a logic program, P , an *SLD derivation* of $P \cup G$ consists of a (possibly infinite) sequence of goals $G_0, G_1, G_2, \dots$, of a sequence $C_1, C_2, \dots$ of clauses in P which have been renamed in such a way as to avoid variable capture and a sequence $\vartheta_1, \vartheta_2, \dots$, of m.g.u. s such that G_0 is G and, for $i \geq 1$, every G_i , is derived from G_{i-1} and C_i using ϑ_i . An *SLD refutation* of $P \cup G$ is a finite SLD derivation of $P \cup G$ which has the empty clause as the last resolvent of the derivation. If $\vartheta_1, \vartheta_2, \dots$ are the m.g.u. s used in the refutation of $P \cup G$, we say that the substitution $\vartheta_1 \vartheta_2 \dots \vartheta_n$ restricted to the variables occurring in G is the *computed answer substitution* for $P \cup G$ (or for the goal G in the program P).

Classic results, due to K. L. Clark, show that this rule is sound and complete with respect to the traditional first-order logical interpretation.

Indeed, it can be proved that if ϑ is the *computed answer substitution* for the goal G in program P , then $G\vartheta$ is a logical consequence of P (soundness). Moreover, if $G\vartheta$ is a logical consequence of P , then no matter which selection rule is used, there exists a SLD refutation of $P \cup G$ with computed answer σ such that $G\sigma$ is more general than $G\vartheta$ (strong completeness).

Heads with Arbitrary Terms

So far, we have assumed that the heads of clauses contain just distinct variables. We have made this choice to preserve the similarity with procedures in traditional languages. However, real logic programs also use arbitrary terms as arguments to predicates in heads, as we have seen in the example in Sect. 12.1.1. The box on Sect. 12.4.3 provides the evaluation rule for such a general case. Note, however, that our assumption is in no way limiting (apart, perhaps, from textual convenience). Indeed, as it is clear from what was said in the box and from the preceding treatment, a clause of the form:

$$p(t_1, \dots, t_n) : -A_1, \dots, A_m$$

can be seen as an abbreviation of the clause:

$$p(X_1, \dots, X_n) : -X_1 = t_1, \dots, X_n = t_n, A_1, \dots, A_m.$$

In the following examples, we will often use the notation with arbitrary terms in heads.

12.4.4 Control: Non-determinism

In the evaluation of a goal, we have two degrees of freedom: the selection of the atom to evaluate and the choice of the clause to apply.

For the first, we have said that we can fix a selection rule, without influencing the final results of the computation that terminate in success. For this reason, the non-determinism arising from the selection of the atom is called “*don’t care*”.

For the selection of clauses, on the other hand, the matter is more delicate. Given that a predicate can be defined by more than one clause, and that we have to use only one of them at a time, we could think about fixing some rule for choosing clauses, analogous to the one used to choose atoms. The following program reveals however a problem. Let $\mathcal{P}a$ be the program:

$p(X) :- p(X).$	1
$p(X) :- X=a.$	2

and let us assume that we choose clauses from top to bottom, following the textual order of the program. It is easy to see that by adopting this rule, the evaluation of $p(Y)$ never terminates and therefore we do not have a computed answer by the program. Indeed, using clause (1) and the substitution $\{X/Y\}$, the initial goal, after one computation step, becomes the goal $p(Y)$, which is again the starting goal. According to the clause-selection rule, we must choose clause (1) and apply it to this (new) goal, and so on. It is however clear that using clause (2) we would immediately obtain a terminating computation, producing the substitution $\{Y/a\}$ as result. Note that to fix another order, for example, from bottom to top, would not in general solve this problem.

In the light of this example, let us carefully reconsider the evaluation rule for a goal seen in the previous subsection and, in particular, let us fix on point 2(a), where we wrote “if there exists a clause” without specifying how this can be chosen. By doing this, we have therefore introduced into the computation model a form of non-determinism: in the case in which there is more than one clause for the same predicate, we can choose one in a non-deterministic fashion, without adopting any particular rule. This form of non-determinism is called “*don’t know*” non-determinism, because we do not know which the “right” clause will be that allows the termination of the computation with success. The theoretical model of logic programming keeps this non-determinism when it considers all possible choices of clause and therefore all possible results of the various computations that are produced as a consequence of this choice. The result of the evaluation of a goal G in the program P is therefore a set of computed answers, where each of these answers is the substitution obtained by the composition of all the m.g.u.s which are encountered in a specific computation (with specific choices of clauses), restricted to the variables present in G . For a more precise definition of this idea, as well as of the whole process of evaluating a goal, see the box on Sect. 6.4.

Turning to our previous example, the only computed answer for the goal $p(Y)$ in program $\mathcal{P}a$ is the substitution $\{Y/a\}$ while, for the same program, the goal $p(b)$ has no computed answer given that all its computations either terminate with failure (when the second clause is used) or do not terminate (when the first clause is used).

Backtracking in PROLOG

When moving from the theoretical model to an implemented language, such as PROLOG, non-determinism must, at some level, be transformed into determinism, given that the physical computing machine is deterministic. This can obviously be done in various ways, and in principle does not cause loss of solutions. For example, we could think of starting k parallel computations when there are k possible clauses for a predicate¹⁹ and therefore consider the results of all the possible computations.

In PROLOG, however, for reasons of simplicity and implementation efficiency, the strategy we first saw is employed: clauses are used according to the textual order in which they occur in the program (top-to-bottom). We saw in the previous example that this strategy is *incomplete*, given that it does allow us to find all possible computed answers. This limit, however, is adjustable by the programmer who, knowing this property of PROLOG, can order the clauses in the program in the most convenient way (typically putting first those relating to terminal cases and then the inductive steps). Note, though, that this trick, at least in principle, eliminates some of the declarative nature of the language, because the programmer has specified an aspect of control.

In addition to infinite computations, there is a second, more important aspect to be considered by adopting the deterministic model of PROLOG and deals with the handing of failures. Let us first see an example. Let us consider the program $\mathcal{P}b$:

$p(X) :- X = f(a).$	1
$p(X) :- X = g(a).$	2

Let us consider the evaluation of the goal $p(g(Y))$. By the PROLOG strategy, clause (1) is chosen, which, using the substitution $\{X/g(Y)\}$, produces the new goal $g(Y) = f(a)$. This fails, given that the two terms in the equation cannot be unified. However, given that there is still one clause to use, it would not be acceptable to terminate the computation by returning a failure. “Backtracking” then occurs, returning to the choice of the clause for $p(g(Y))$ and therefore continues the computation by trying clause (2). In this way, the computation achieves success with the computed answer $\{Y/a\}$.

In general, therefore, when arriving at a failure, the PROLOG abstract machine “backtracks” to a previous choice point at which there are other choices; that is, where there are other clauses to test. In this backtracking process, the bindings that might have been computed in the previous computation must be undone. Once a choice point has been reached, a new clause is tried and the computation continues in the way we have seen. If the previous choice point contains no possibilities for computations,

¹⁹ Clearly, on a machine with one processor, the k parallel computations must be appropriately “scheduled” so they can be executed in a sequential manner, analogous to what happens with processes in a multitasking operating system.

an older choice point is sought and, if there are no more, the computation terminates in failure. Note that all of this is handled directly by the PROLOG abstract machine and is completely invisible to the programmer (except the use of particular constructs such as the cut which we will introduce in Sect. 12.5.1).

It is also easy to see how this procedure, which uses a search corresponding to a depth-first search through a tree representing the possible computations, can be computationally demanding. The solution to the problem seen in Sect. 12.1.1, for example, requires extensive backtracking and is fairly wasteful in computation time.

Let us now see another example. Consider the program `Pc`:

```

p ( X ) : -   X = f ( Y ) ,   q ( X ) .           1
p ( X ) : -   X = g ( Y ) ,   q ( X ) .           2
q ( X ) : -   X = g ( a ) .                       3
q ( X ) : -   X = g ( b ) .                       4

```

Let us analyse the goal $p(Z)$. Using clause (1), we obtain the goal $Z = f(Y), q(Z)$. The evaluation of the equation produces the m.g.u. $\{Z/f(Y)\}$ and we obtain the goal $q(f(Y))$. Using clause (3), we obtain the goal $f(Y) = g(a)$, which fails. At this point we must turn to the last choice point, that is to the point where the choice of clause for the predicate q occurred. In this case, there are no bindings to undo, and therefore we try clause (4), obtaining therefore the goal $f(Y) = g(b)$ which also fails. We return again to the choice point for q and we see that there are no other possible clauses and therefore we return to the previous choice point (the one for predicate p). Doing this, we have to undo the binding $\{Z/f(Y)\}$ calculated by clause (1) and therefore we return to the initial situation, where variable Z is not instantiated. Using clause (2), we obtain the goal $Z = g(Y), q(Z)$ and therefore by the evaluation of the equation, we obtain the m.g.u. $\{Y/a\}$ and the new goal $q(g(Y))$. At this point, using clause (3), we obtain the goal $g(Y) = g(a)$ which succeeds and produces the m.g.u. $\{Y/a\}$. Given that there remain no more goals to evaluate, the computation terminates with success. The result of the computation is produced by composing the computed m.g.u. $s\{Z/g(Y)\}\{Y/a\}$ and restricting the substitution $\{Z/g(a), Y(a)\}$ obtained by this composition to the single variable present in the initial goal. Therefore, the computed answer obtained is $\{Z/g(a)\}$. Note that there is another computed answer, $\{Z/g(b)\}$, which we can obtain using clause (4) rather than (3). In PROLOG implementations, answers subsequent to the first can be obtained using the “;” command. Finally, the reader can easily check that the goal $p(g(c))$ in program `Pc` terminates with failure, a result that is obtained after having proved all four of the possible clause combinations.

12.4.5 Some Examples

In this subsection, we will focus on the PROLOG language, and we will make use of its syntax. The notation $[h|t]$ is used to denote the list which has h as its head and t as its tail. Let us remember that the head is the first element in the list, while the tail is what remains of the list when the head has been removed. The empty list is written $[]$, while $[a,b,c]$ is an abbreviation for

`[a | [b | [c | [] ] ] ]` (the list composed of elements `a`, `b` and `c`). Note that in PROLOG, as in pure logic programming language, there exists no list type, for which the binary function symbol `[ | ]` can be used for terms that are not lists. For example, we can also write `[a | f(a)]` which is not a list (because `f(a)` is not a list).

As the first example, let us consider the following program `member` which checks whether an element belongs to a given list:

```
member(X, [X | Xs]).
member(X, [_ | Xs]) :- member(X, Xs).
```

The declarative reading of the program is immediate: Clause (1) consists of the terminal case in which the element that we are looking for (the first argument of the `member` predicate) is the head of the list (the second argument to the `member` predicate). Clause (2) instead provides the inductive case and tells us that `X` is an element of the list `[Y|Xs]` if it is an element of the list `Xs`.

Formulated in this way, the `member` program is similar to the one that we could write in any language that supports recursion. However, let us note that, unlike in the imperative and functional paradigms, this program can be used in two different ways.

The more conventional mode is the one in which it is used as a test. In a PROLOG system, once the previous program has been input, we have:

```
?- member(hewey, [dewey, hewey, louie]).
Yes
```

where `?-` is the abstract machine's prompt, to which we have added the goal whose evaluation we require. The next line contains the interpreter's answer. In this case, we have a simple "boolean" answer which expresses the existence of a successful computation of our goal. Moreover, as we know, we can also use the program to compute. For example, we can ask for the evaluation of:

```
?- member(X, [dewey, hewey, louie]).
X = dewey
```

The abstract machine returns `{X/dewey}` as the computed answer. We can also obtain more answers using the `“;”` command. When there are no more answer, the system replies `“no”`.

Finally, even if it is used in a rather unnatural fashion, we can use the first argument to instantiate the list that appears as the second argument. For example:

```
?- member(dewey, [X, hewey, louie]).
X = dewey
```

This possibility of using the same arguments as inputs or as outputs according to the way in which they are instantiated is unique to the logic paradigm and is due to the presence of unification in the computational model.

We can clarify this point further by considering the following `append` program which allows us to concatenate lists:

```
append([], Ys, Ys).                                     1
append([X|Xs], Ys, [X|Zs]) :- append(Xs, Ys, Zs).      2
```

In this case, too, the declarative reading is immediate. If the first list is empty, the results is the second list (clause(1)). Otherwise, (inductively), if Zs is the result of the concatenation of Xs and Ys , the result of the concatenation of $[X|Xs]$ and Ys is obtained by adding X to the head of the list Zs , as indicated by $[X|Zs]$.

The normal use of the `this` program is illustrated by the following goal:

```
?- append([dewey, hewey], [louie, donald], Zs).
Zs = [dewey, hewey, louie, donald]
```

Moreover, we can use `append` also to know how to subdivide a list into sublists, something that is not possible in a functional or imperative program:

```
?- append(Xs, Ys, [dewey, hewey]).
Xs = []
Ys = [dewey, hewey];
Xs = [dewey]
Ys = [hewey];
Xs = [dewey, hewey]
Ys = [];
no
```

Here, as before, the “;” command causes the computation of new solutions.

As a third example, we give the definition of the `sublist` predicate (we used this in Sect. 12.1.1). If Xs is a sublist of Ys , then there exist another two (possibly empty) lists As and Bs such that Ys is the concatenation of As , Xs and Bs . We note that this means that Xs is the suffix of a prefix of Ys . Hence using `append`, we can define `sublist` as follows:

```
sublist(Xs, Ys) :- append(As, XsBs, Ys), append(Xs, Bs, XsBs).
```

This program, combined with the `append` program and the `sequence` program of Sect. 12.1.1 allow us to solve the problem stated in Sect. 12.1.1. Note the conciseness and simplicity of the resulting program. Clearly other definitions of `sublist` are possible (see Exercise 8).

As a last example, let us consider a program that solves the classical problem of the Towers of Hanoi. We have a tower (in Computer Science terms, we would say a stack) composed of n perforated discs (of different diameters), arranged on a pole in order of decreasing diameter and we have 2 free poles. The problem consists of moving the tower to another pole, recreating the initial order of the discs. The following rules apply. The disks can be moved only from one pole to another. Only one disc at a time can be moved from one pole to another and the top disc must be taken from a pole. A disc cannot be put onto a smaller disc.

According to legend, this problem was assigned by the Divinity to the monks of a monastery near to Hanoi. There were three poles and 64 discs of gold. The solution to the problem would have signalled the end of the world. Given that the optimal solution requires time exponential in the number of discs, even if the legend comes true, we can still remain calm for a while: 2^{64} is a large enough number.

The following program solves the problem for an arbitrary number of disc, N , and three poles called A , B and C . It uses the coding of the natural numbers in terms of 0 and successor that we saw above.

```
hanoi(s(0), A, B, C [move(A,B)]).
```

```
hanoi(s(N), A, B, C, Moves):-
    hanoi(N, A, C, B, Moves1),
    hanoi(N, C, B, A, Moves2),
    append(Moves1, [move(A,B)|Moves2], Moves).
```

The call `hanoi(n, A, B, C, Move)`, where `n` is a (term which represents a) natural number, solves the problem of moving the tower of `n` discs from `A` to `B` using `C` as an auxiliary pole. The solution is contained in the `Move` variable which, when the computation terminates, is instantiated with the list containing move which contribute to the solution. Every move is represented by a term of the form `move(X, Y)` to indicate the move of the top disc of pole `X` to pole `Y`.

This amazingly simple program demonstrates the power of logic programming and recursive reasoning. Let us give its declarative interpretation. The first clause is clear: if we have just one disc, all we have to do is to move it from `A` from `B`. The reading of the second clause also is quite intuitive. If `Moves1` is the list of moves that solves the problem of moving a tower of `N` discs from `A` to `C` using `B` as an auxiliary pole and `Moves2` is the list of moves needed to move the tower of `N` disc from `C` to `B` using `A` as an auxiliary pole, to solve our problem in the case of `N+1` (or `s(N)`) discs, we have to do the following: first, execute all the moves in `Moves1`, then move from `A` to `B` (with the move `move(A, B)`) the `N+1`st disc in `A` which, being the largest, must be the last (at the bottom) in `B` as well. Finally, execute all the moves in `Moves2`. This is what is done by the `append` predicate which therefore will use the variable `Moves` to provide the solutions to this problem.

12.5 Extensions

Until now, we have seen pure logic programming and some, highly partial, aspects of PROLOG. In this last section, we will briefly outline some of the numerous extensions to the pure formalism which are used in real implemented logic languages. We defer to the next chapter an important extension of logic programming which is nowadays used in most PROLOG languages, namely constraint logic programming. In the bibliographical notes we have included some references for those wishing to learn more.

12.5.1 Prolog

PROLOG is a language that is much richer than it might appear from what has been said so far. Let us recall that this language differs from logic programming in its adoption of precise rules for selecting the next atom to be rewritten (left to right) and for selecting the next clause to use (top to bottom, according to the program text).

In addition to these, there are other important differences with the theoretical formalism. We will list some of these without pretending to be exhaustive.

Arithmetic

A real language cannot allow itself the luxury of using a completely symbolic arithmetic with no predefined operators. In PROLOG, there exist therefore integers and reals (as floating point) as predefined data structures and various operators for manipulating them. These include:

- The usual arithmetic operators $+$, $-$, $*$ and $//$ (integer division).
- Arithmetic comparison operators such as $<$, $<=$, $>=$, $==$ (equal), $= \setminus =$ (different).
- An evaluation operator called `is`.

Unlike other (function and predicate) symbols used in PROLOG, these symbols use infix notation for ease of use. There are however numerous delicate aspects that should be known to avoid simple mistakes.

First, comparison operators always require that their operands are ground arithmetic expressions (they cannot contain variables). Thus, while we have²⁰:

```
?- 3*2 == 1+5
Yes
? 4 > 5+2
no
```

if we use terms that are not arithmetic expressions or which contain variables, error will be signalled, as in:

```
?- 3 > a
error in arithmetic expression: a is not a number
?- X == 3+5
instantiation fault.
```

The last example is particularly disturbing. The evaluation of the arithmetic expression $3+5$ produces the value 8 which we might expect to be bound to the variable X . This cannot be done using $==$, nor can it be done using syntactical equality between terms as considered in the previous paragraph. If we write $X = 3+5$, indeed, the effect is that of binding X to the term $3+5$ (rather than the value 8). To avoid this problem, the expression evaluation operator `is` is introduced. This operator allows us to obtain the effect we desire; $s \text{ is } t$ indeed *unifies* s with the *value* of the ground arithmetic expression t (if t is not a ground arithmetic expression, we have an error). The following are some examples of the use of `is`:

²⁰ At the prompt `?-` we find the query that we want to evaluate; the following line is the PROLOG interpreter's reply.


```

?- X is 3+5;
X = 8
?- 8 is 3 + 5
Yes
?- 6 is 3 * 3
no
?- X is Y*2
error in arithmetic expression: Y*2 is not a number

```

The use of `is`, necessary to be able to evaluate an expression, makes the use of arithmetic in PROLOG rather complicated. For example, given the following program:

```

evaluate(0,0).
evaluate(s(X), Val+1) :- evaluate(X, Val)

```

The goal `evaluate(s(s(s(0))), X)` does not compute the value 3 for `X`, as perhaps would have been expected, but the term `0+1+1+1` (see Exercise 9 at the end of this chapter, too).

Cut

PROLOG provides various constructs for interaction with the abstract machine's interpreter so that the normal flow of control can be modified. Among these, one of the more important (and most discussed) is *cut*. This is an argument-free predicate, written as an exclamation mark, which allows the programmer to eliminate some of the possible alternatives produced during evaluation, with the aim of increasing execution efficiency. It is used when we are sure that, when a condition is satisfied, the other clauses in the program are no longer useful. For example, the following program computes the minimum of two value. If a comparison condition is true, the other is necessarily false, so we can use `!` to express the fact that once the first clause has been used, there is no need to consider the second.

```

minimum(X, Y, X) :- X < Y, !.
minimum(X, Y, Y) :- X > Y.

```

Alternatively, if we are interested only in testing whether a value appears at least once in a list, we can use the following modification to the `member` program:

```

member(X, [X | Xs]) :- !.
member(X, [_ | Xs]) :- member(X, Xs).

```

In general, the meaning of `cut` is the following. Suppose that we have n clauses defining the predicate p

```

p(S1) :- A1.
...
p(Sk) :- B, !, C.
...
p(Sn) :- An.

```

If, during the evaluation of the goal $p(t)$, we find the k th clause in the list being used, we have the following cases:

1. If an evaluation of B fails,²¹ then we proceed by trying the $k + 1$ st clause.
2. If, instead, the evaluation of B succeeds, then $!$ is evaluated. It succeeds (it always does) and the evaluation proceeds with C . In the case of backtracking, however, all the alternative ways of computing B are eliminated, as well as are eliminated all the alternatives provided by the clauses from the k th to the n th to compute $p(\tau)$.

There is no need to say that cut, in addition to not being easy to use, eliminates a good part of the declarativeness of a program.

Disjunction

If we want to express the disjunction of two goals, $G1$ and $G2$, that is the fact that it is sufficient that at least one of the two succeeds, we can use two clauses of the form:

```
p(X) :- G1.
p(X) :- G2.
```

with the goal $p(X)$. The same effect can be obtained writing $G1 ; G2$, where we have used the predefined predicate “;”, which represents disjunction.

If-then-else

The traditional construct from imperative languages:

```
If B then C1 else C2
```

is provided in PROLOG as a built-in with syntax $B \rightarrow C1 ; C2$. This construct is implemented using cut as follows:

```
if_then_else(B, C1, C2) :- B, !, C1.
if_then_else(B, C1, C2) :- C2.
```

Negation

Thus far, we have seen that in the body of a clause, only “positive” atomic formulae can be used; in other words, they cannot be negated.²² Often, however, it can also be useful to use negated atomic formulae. For example, let us consider the following program, flights:

```
direct_flight(bologna, paris).
direct_flight(bologna, amsterdam).
direct_flight(paris, bombay). 2
```

²¹ Here, B , just like A_i and C , is considered as an arbitrary goal and not necessarily an atomic one.

²² Here, we refer to the $H : -A_1, \dots, A_n$ notation. If, instead, we consider a clause as a disjunction of literals, the atoms in the body are negated, given that the preceding representation is equivalent to $\neg A_1 \vee \dots \vee \neg A_n \vee H$.

```

direct_flight(amsterdam, moscow).           4
flight(X, Y):- direct_flight(X, Y).
flight(X, Y):- direct_flight(X, Z), flight(Z, Y). 6

```

Here, we have a series of facts defining the `direct_flight` predicate, which denotes the existence of a connection without stopover between two destinations. Then, we have the `flight` predicate which defines a flight, possibly with intermediate stops, between two locations. This will be a direct flight (clause 5) or rather a direct flight for a stopover different from the one that is the destination, followed by a flight from this stopover to the destination (clause 6). With this program, we can check the existence of a flight, or we can ask which destinations can be reached from a given airport:

```

?- flight(bologna, bombay)
Yes
?- direct_flight(bologna, moscow)
no
?- flight(bologna, X)
X = paris

```

However, we fail to express the fact that there exists only non-direct flights. Indeed, there is no way we can express that there *does not* exist a direct flight. To do this, we need negation. If we write:

```
indirect_flight(X, Y) :- flight(X, Y), not direct_flight(X, Y).
```

we mean to say that there exists an indirect flight between `X` and `Y` if there exists a flight (between `X` and `Y`) and there exist no direct flight. With this definition, we have:

```

?- indirect_flight(bologna, bombay)
Yes
?- indirect_flight(bologna, paris)
no

```

These results can be explained as follows. The PROLOG interpreter evaluates a goal such as `not G` by trying to evaluate the un-negated goal `G`. If the evaluation of this goal terminates (possibly after backtracking) with failure, then the goal `not G` succeeds. If, on the other hand, the goal `G` has a computation that terminates with success, then that of `not G` fails. Finally, if the evaluation of `G` does not terminate, then `not G` fails to terminate. This type of negation is called “negation as failure”,²³ in that it interprets the negation of a goal in terms of failure of the un-negated goal. Note that, because of infinite computations,²⁴ this type of negation differs from the negation in classical logic, given that the lack of success of `G` is not equivalent to the success of the negated version `not G`. For example, neither the goal `p` nor the goal `not p` succeed in the program `p :- p`.

²³ To be more precise, negation as failure, as defined in the theoretical model of logic programming, has a behaviour that is slightly different from that described because of the incompleteness of the PROLOG interpreter.

²⁴ And also non-ground goals, which we will not consider here.

12.5.2 Logic Programming and Databases

The `flight` program above indirectly indicates a possible application of logic programming in the context of databases.

A set of unit clauses, such as those defining the `direct_flight` predicate, is indeed, to all effects, the explicit (or extensional) definition of the relation denoted by this predicate. In this sense, this set of unit clauses can be seen as the analogue of a relation in the relational database model. To express a query, while in the relational model we would use relational algebra with the usual operations of selection, projection, join, etc., or more conveniently, a data manipulation language such as SQL, in the logic paradigm we can use its usual computational mechanisms. For example, to find out if there is a direct flight to Paris, we can use the query `direct_flight(X,paris)`. The predicates defined by the non-unit clauses such as `flight`, define relations in an implicit (or intensional) manner. They, indeed, allow us to compute new relations which are not explicitly stored, through the mechanism of inference. Using database terminology, we will say that these predicates define “views” (or virtual relations).

Note, moreover, that in the `flights` program, function symbols are not used. Indeed, if we want only to manipulate relations, as is the case in relational algebra, function symbols are not needed.

These considerations were crystallised in the definition of *Datalog*, a logic language for databases. In its simplest form, it is a simplified version of logic programming in which:

1. There are no function symbols.
2. Extensional and intensional predicates are distinguished, as are comparison predicates.
3. Extensional predicates cannot occur in the head of clauses with a non-empty body.
4. If a variable occurs in the head, it must occur in the body of a clause.
5. A comparison predicate can occur only in the body of a clause. The variables occurring in such a predicate must occur also in another atom in the body of the same clause.

The last two conditions deal with the specific evaluation rule which Datalog uses²⁵ and will here will be ignored. The distinction between extensional and intensional predicates corresponds to the intuitive idea that has already been discussed. Extensional predicates, as condition (3) implies, can be defined only by facts.

The `flights` program can therefore be considered as a Datalog program.

The reader who knows SQL or relational algebra will have no difficulty in understanding how the presence of recursion increases the expressive power of Datalog beyond that of these other formalisms. Our program, `flights`, provides us with

²⁵ Datalog adopts a kind of bottom-up evaluation mechanism for goals. This differs from the top-down one we saw for logic programming. The results obtained are, however, the same.

an example in this sense: the query `flight(bologna, X)` allows us to find all the destinations that can be reached from Bologna with an arbitrary number of intermediate stops. This query is not expressible in relational algebra or in SQL (at least, in its initial version) since the number of joins that we must create depends on the number of intermediate stops and, in general, not knowing how many of them there are (we are making no assumptions about how relations are structured), we cannot define such a number *a priori*.

12.6 Advantages and Disadvantages of the Logic Paradigm

The few examples given in this chapter should be sufficient to show that logic languages require a programming style that is significantly different from that of traditional languages. They also have a different expressive power. Clearly, since both PROLOG and the conventional programming languages are Turing-complete formalisms, we are not saying that logic languages allow “to compute more things” than common imperative languages. The expressive power to which we refer is of a pragmatic kind, as we will make clearer in this section.

First, it is worth repeating that logic languages, in a way analogous to functional languages, allow us express solutions even to very complex problems in a “purely declarative” way, and therefore in a simple and compact way. In our specific case, these aspects are further reinforced by three unique properties of the logic paradigm: (i) the ability to use a program in more than one way by transforming input arguments into outputs and viceversa; (ii) the possibility of obtaining a complex relationship between variables as a result, which implicitly expresses an infinite number of solutions (given by all the values of the variables that satisfy the relation); (iii) the possibility of reading a program directly as a logical formula.

The first characteristic derives from the fact that the basic computational mechanism, unification, is intrinsically bi-directional. This does not happen either with assignment in imperative languages, or with pattern matching in functional languages because both of these mechanisms presuppose a direction in variable bindings. This flexibility clearly allows us to use every program to its best, avoiding the duplication of code for modifications that are often only of a syntactic nature.

The second property is particularly important to all application areas in which a single solution, understood as a set of specific values of interesting variables, either cannot be obtained (because the data do not contain enough information), or is not interesting, or is particularly difficult to obtain computationally. This property will be more evident in the next chapter, when will discuss constraint logic programs .

The third aspect, finally, at least in principle, makes it easier to verify the correctness of logic programs than imperative ones, where a logical reading of programs is possible but requires much more expressive (and complicated) tools. As already emphasised in Sect. 11.4, this aspect is essential to be able to obtain formal correctness-verifying tools that are easily usable on programs of significant size.

These three aspects, which we can summarise in the principle of “computation as deduction”, allow us to express in a natural fashion forms of reasoning that we encounter in artificial intelligence and knowledge representation (for example, in the context of the Semantic Web). The fact that control is handled entirely by the abstract machine through backtracking, allows to easily express search problems in a space of solutions.

Logic languages can also profitably be used as tools for the rapid prototyping of systems. Using PROLOG, it is possible to describe in a very short period and in a compact way, even very complex systems, with the advantage, as far as specification languages are concerned, that it can be executed.

Thus far, advantages. Beside them, however, we can record various negative aspects of logic languages, at least in their classical versions.

First, as already noted in this chapter, the control mechanism based on backtracking is of limited efficiency. Various expedients can be used to improve this aspect, be it at the level of constructs used in programs, or at the implementation level. For example, abstract machines specific to logic languages such as the WAM (Warren Abstract Machine) have been defined and have been variously optimised. However, the efficiency of a PROLOG program remains fairly limited.

The absence of types or modules is a second bad point, even if in this sense some more recent languages have proposed partial solutions (for example, the logic language, Mercury). From what was said in Chap. 8, it should be clear that the lack of an adequate type system makes program correctness checking relatively difficult, despite the possibility of declarative reading.

Also, the arithmetic constructs and, in general, built-ins in PROLOG certainly do not facilitate the correctness of programs. This is because their use is not easy due to various semantic subtleties. This is particularly evident in the control constructs, which frequently force us to choose between a clear program that is not efficient and an efficient one that is, however, hard to understand.

Finally, and for many reasons (not just a limited commercial interest), logic languages almost always have a programming environment that is rather deficient, where there are few of the characteristics available in the sophisticated environments for object-oriented programming.

12.7 Summary

In this chapter, we have presented the primary characteristics of the logic programming paradigm, a relatively recent paradigm (which has developed since the mid-1970s), which implements the old aspiration of seeing computation as a process that is entirely governed by logical laws. With only one chapter available to us, we have had to limit ourselves to introductory aspects, however what has been presented has enough formal precision to illustrate the computational model for logic programming languages. In particular, we have seen:

- Some syntactic concepts in first-order logic which are necessary for the definition of clauses and therefore logic programs.
- The process of unification which comprises the basic computational mechanism. This required a few concepts relating to terms and substitutions. We have also seen a specific algorithm for unification.
- How computation in a logic language works using a rule of deduction called SLD Resolution. In the text, we have chosen an approach that was inspired by a procedural reading of the clauses that demonstrate the similarity with “normal” procedures in imperative languages. Two boxes more formally introduced the concepts.

After some examples, we saw some important extensions to the pure formalism, in particular:

- Some specific characteristics of the PROLOG language. We did not cover meta-programming, higher-order programming or constructs that manipulate the program at runtime, all of which are very important.
- The idea of the use of logic languages in connection with databases.

For more information on those aspects of PROLOG that we have omitted or for a better introduction to Datalog, the reader is recommended to the literature listed below. On the other hand Constraint logic programs, an important extension, will be considered in the next chapter.

12.8 Bibliographical Notes

Herbrand’s ideas on unification are in his doctoral dissertation from 1930 [2]. The definition of the resolution rule and the first formalisation of the unification algorithm are in A. Robinson’s work [3]. The unification algorithm that we have presented was introduced in [4]. Major details of theory of unification can be obtained from various articles and texts, among which is [5].

The “historic” paper by R. Kowalski which introduced SLD Resolution and therefore the theory of logic programming is [6]. K. L. Clark’s results on correctness and completion are in [7].

There are many texts both on the theory of logic programming and on programming in PROLOG. Among the former, there is the text by Lloyd [8] and the more recent and detailed treatment by Apt [1]. For programming in PROLOG and for numerous (non-trivial) examples of programming, we advise the reader to consult the classic text by Sterling and Shapiro [9], from which we have taken the program that solves the problem of the Towers of Hanoi; there is also the book by Coelho and Cotta [10], from which we have taken the program in the example presented in Sect. 12.1.1.

Finally, as for Datalog and, more generally, the use of logic programs with databases, [11] can be consulted.

12.9 Exercises

1. State a context-free grammar which defines propositional formulæ (that is those obtained by considering only predicates of arity 0 and without quantifiers).
2. Assume that we have to represent the natural numbers using 0 for zero and $s(n)$ for the successor of n . State what is the computed answer obtained by evaluating the goal $p(s, t, X)$, in the following logic program, where s and t are terms that represent natural numbers:

```
p(0, X, X).
p(s(Y), X, s(Z)):- p(Y, X, Z).
```

3. State what are the computed answers obtained by evaluating the goal $p(X)$ in the following logic program:

```
p(0).
p(s(X)):- q(X).
q(s(X)):- p(X).
```

4. Given the following logic program:

```
member(X, [X| Xs]).
member(X, [_| Xs]):- member(X, Xs).
```

State what is the result of evaluating the goal:

```
member(f(X), [1, f(2), 3]).
```

5. Given the following PROLOG program (recall that X and Y are variables, while a and b are constants):

```
p(b):- p(b).
p(X):- r(b).
p(a):- p(a).
r(Y).
```

State whether the goal $p(a)$ terminates or not; justify your answer.

6. Consider the following logic program:

```
p(X):- q(a), r(Y).
q(b).
q(X):- p(X).
r(b).
```

State whether the goal $p(b)$ terminates or not; justify your answer.

7. Assuming that the natural numbers are represented using 0 for zero and $s(n)$ for the successor of n and using a primitive `write(x)` that writes the term t , write a logic program that prints all the natural numbers.
8. Define the `sublist` predicate in a direct fashion without using `append`.
9. Write in PROLOG a program that computes the length (understood as the number of elements) of a list and returns this value in numeric form. (Hint: consider an inductive definition of length and use the `is` operator to increment the value in the inductive case.)

10. List the principle differences between a logic program and a PROLOG program.
11. If in a logic program, the order of the atoms in the body of a clause is changed, is the semantics of the program altered? Justify your answer.
12. If in PROLOG, the clause-selection rule were changed (for example, always selecting the lowest instead of the highest) would it change the semantics of the language? Justify your answer.
13. Give an example of a logic program, P , and of a goal, G , such that the evaluation of G in P produces a different effect when two different selection rules are used. (Suggestion: since for computed answers there is no difference in the use of different selection rules, consider what happens to computations that do not terminate and that fail.)
14. Informally describe a selection rule that allows us to obtain the least possible number of computations that do not terminate. (Suggestion: computations that do not terminate, by changing the selection rule, would become finite failures. Consider a rule that guarantees that all atoms in the goal are evaluated.)

References

1. K.R. Apt, *From Logic Programming to Prolog* (Prentice Hall, 1997)
2. J. Herbrand, *Logical Writings* (Reidel, Dordrecht, 1971)
3. J.A. Robinson, A machine-oriented logic based on the resolution principle. *J. ACM* **12**(1), 23–41 (1965)
4. A. Martelli, U. Montanari, An efficient unification algorithm. *ACM Trans. Program. Lang. Syst.* **4**, 258–282 (1982)
5. E. Eder, Properties of substitutions and unifications. *J. Symb. Comput.* **1**, 31–46 (1985)
6. R.A. Kowalski, Predicate logic as a programming language. *Inf. Process.* **74**, 569–574 (1974)
7. K.L. Clark, Predicate logic as a computational formalism. Technical Report Res. Rep. DOC 79/59, Imperial College (Department of Computing, London, 1979)
8. J.W. Lloyd, *Foundations of Logic Programming*, 2nd edn. (Springer, 1987)
9. L. Sterling, E. Shapiro, *The Art of Prolog* (MIT Press, 1986)
10. H. Coelho, J.C. Cotta, *Prolog by Example* (Springer, 1988)
11. S. Ceri, G. Gottlob, L. Tanca, *Logic Programming and Databases* (Springer, 1989)